



東莞理工學院
DONGGUAN UNIVERSITY OF TECHNOLOGY

碩士研究生學位論文

(全日制學術型碩士研究生)

題目：分布式存儲系統中 Reed-
Solomon 碼快速解碼算法研究

姓 名：林澤冰

學 號：221115215

學 院：計算機科學與技術學院
(軟件學院，網絡空間安全學院)

學科專業：計算機科學與技術

研究方向：計算機網絡與信息安全

導師姓名：侯韓旭 教授



二〇二五年五月

Research on Fast Decoding Algorithm of Reed-Solomon Codes in Distributed Storage Systems

A Dissertation Submitted for the Degree of Master

Candidate: Lin Zebing

Supervisor: Prof. Hou Hanxu

Speciality: Computer Science and Technology

**Research field: Computer Network and
Information Security**

Dongguan University of Technology

Dongguan, China

分类号：TP301

学校代号：11819

学 号：221115215

东莞理工学院学术型硕士研究生学位论文

**（分布式存储系统中 Reed-Solomon 码快速
解码算法研究）**

作者姓名：林泽冰

导师姓名、职称：侯韩旭、教授

学科专业：计算机科学与技术

研究方向：计算机网络与信息安全

论文答辩日期：2025 年 5 月 28 日

论文提交日期：2025 年 5 月 27 日

答辩委员会成员：

主席：张涵

委员：夏隽娟、赖颖昕、张雁峰、孙泽文

摘要

当前,云计算等技术飞速发展产生了海量数据,这些海量数据对存储系统的可靠性和存储效率提出了更高的要求。最大距离可分码(Maximum Distance Separable Codes, MDS Codes)因容错能力与存储开销可以达到最佳平衡而被广泛应用在分布式存储系统中。其中,里德-所罗门码(Reed-Solomon Codes, RS Codes)是一类常见的构造在有限域上的 MDS 纠删码,然而其解码过程中需要进行大量有限域的乘法操作,由于乘法操作计算复杂度高,造成了 RS 码的性能瓶颈。针对此问题,研究人员提出了许多快速算法,但在现有的 RS 码解码算法中,如基于里德-穆勒(Reed-Muller, RM)变换的算法在解码时仍需要进行较多的乘法操作,为进一步减少 RS 码解码过程所需乘法操作次数,降低计算复杂度,本文对 RS 码快速解码算法进行了研究,具体内容如下:

(1) 基于分圆陪集构造了一种范德蒙矩阵,并将其作为校验矩阵设计了一种新的 RS 码结构,进一步提出了基于范德蒙矩阵分解的 RS 码快速解码算法。与现有解码过程需要乘法次数最少的基于 RM 变换解码算法相比,当码字长度 $N=16$,校验长度 $T=4$ 和 $T=7$ 时,本文提出的算法在解码过程中需要的乘法次数分别减少 7%和 29%。

(2) 在服务器中搭建 Hadoop 分布式存储系统。在系统上测试本文提出的基于范德蒙矩阵分解的 RS 码不同参数下的性能表现,并与 RM 算法、ISA-L、Jerasure2.0 进行了比较。结果显示,当码字长度 $N=256$,校验长度 $T=16, 32, 64, 128$ 时,本文提出算法解码速度相比 RM 算法、ISA-L 和 Jerasure2.0,分别提升了 5%~17%、68%~271%和 504%~976%。

关键词: 分布式存储, Reed-Solomon 码, 范德蒙矩阵, 纠删码

ABSTRACT

With the rapid development of technologies such as cloud computing, massive amounts of data are being generated, imposing higher requirements on the reliability and storage efficiency of storage systems. Maximum Distance Separable Codes (MDS Codes), which achieve optimal balance between fault tolerance and storage overhead, are widely used in distributed storage systems. Reed-Solomon (RS) codes are a common class of MDS erasure codes constructed over finite fields. However, the decoding process of RS codes requires a large number of finite field multiplications, which have high computational complexity and lead to performance bottlenecks. To address this issue, researchers have proposed various fast algorithms. However, in existing RS decoding algorithms, such as the Reed-Muller (RM) transform decoding algorithm, a significant number of multiplication operations are still required during decoding. To further reduce the number of multiplications required in RS decoding and lower computational complexity, this thesis studies fast RS decoding algorithms. with the following specific contributions:

(1) A Vandermonde matrix is constructed based on cyclotomic cosets and used as a parity-check matrix to design a new RS code structure. Furthermore, a fast decoding algorithm based on the decomposition of Vandermonde matrix is proposed. When the codeword length $N=16$, compared to the existing RM transform decoding algorithm, which requires the fewest multiplications, the proposed algorithm reduces the number of multiplications by 7% and 29% when the number of parities $T=4$ and $T=7$, respectively.

(2) A Hadoop distributed storage system is deployed on a server to evaluate the performance of the proposed algorithm based on the decomposition of Vandermonde matrix with different parameters. The results are compared with RM transform decoding algorithm, ISA-L and Jerasure2.0. The experimental results show that when $N=256$, $T=16, 32, 64$ and 128 , the decoding speed of the proposed algorithm shows improvements of 5%~17%, 68%~271%, and 504%~976% compared to RM transform decoding algorithm, ISA-L and Jerasure 2.0, respectively.

KEY WORDS: Distributed storage, Reed-Solomon codes, Vandermonde matrix, Erasure codes

目 录

第一章 绪论.....	1
1.1 研究背景及意义.....	1
1.2 存储技术方案.....	2
1.3 分布式存储系统.....	3
1.3.1 谷歌文件系统.....	3
1.3.2 Hadoop 分布式存储系统.....	4
1.4 分布式存储系统中的容错技术	5
1.4.1 多副本策略.....	5
1.4.2 纠删码策略.....	6
1.5 国内外纠删码研究现状.....	7
1.5.1 Reed-Solomon 码.....	8
1.5.2 再生码.....	8
1.5.3 阵列码.....	9
1.5.4 非 MDS 码	11
1.6 研究内容与章节安排.....	11
1.6.1 研究内容	11
1.6.2 章节安排.....	12
第二章 Reed-Solomon 码概述	13
2.1 有限域.....	13
2.1.1 有限域的定义及构造.....	13
2.1.2 有限域的计算.....	15
2.2 Reed-Solomon 码构造.....	16
2.2.1 基于生成矩阵的 Reed-Solomon 码构造	16
2.2.2 基于校验矩阵的 Reed-Solomon 码构造	18
2.3 本章小结.....	20
第三章 Reed-Solomon 码解码算法	21
3.1 基础理论介绍.....	21
3.1.1 多点求值与归一化循环卷积.....	21
3.1.2 一种新的 Reed-Solomon 码编码结构	22
3.2 范德蒙矩阵分解.....	23
3.3 快速解码算法.....	29
3.4 解码算法复杂度分析.....	35
3.5 本章小结.....	38
第四章 系统的设计与实现	40

4.1 系统简介.....	40
4.2 系统工作流程.....	41
4.2.1 上传文件.....	41
4.2.2 下载文件.....	43
4.2.3 执行文件编解码算法.....	44
4.3 实验测试.....	45
4.3.1 实验内容说明.....	45
4.3.2 实验环境配置.....	46
4.4 实验结果分析.....	47
4.4.1 编码性能.....	47
4.4.2 解码性能.....	49
4.5 本章小结.....	50
第五章 总结与展望	51
5.1 本文总结.....	51
5.2 未来展望.....	51
参考文献.....	53

第一章 绪论

1.1 研究背景及意义

在数字产业化、工业互联网、大数据等技术飞速发展的今天，一方面人们生活水平得以提升，另一方面数据的产生和累积^[1-3]也在不断加快。这些海量数据融入到人们生产生活的方方面面，对经济社会的运行和治理产生了深刻的影响，根据 IDC 的预测，2024 年全球产生 159.2ZB 的数据量，并且到 2028 年，这一数字将达到 384.6ZB，其复合增长率为 24.4%^[4]。就国内而言，由国家互联网信息办公室发布的《国家信息化发展报告（2023）》^[5]，2023 年我国的数据生产总量达到 32.85ZB，同比增长超过 22%，数据产业规模 1.74 万亿元。这些海量数据的产生催动了存储行业的发展，以我国数据中心的标准机架数量为例，其数量由 2019 年的 315 万架持续增长到 2023 年的 810 万架^[6]。此外，据赛迪顾问预测，国内分布式存储市场规模将逐年上涨，并在 2026 年达到 255.9 亿元^[7]。

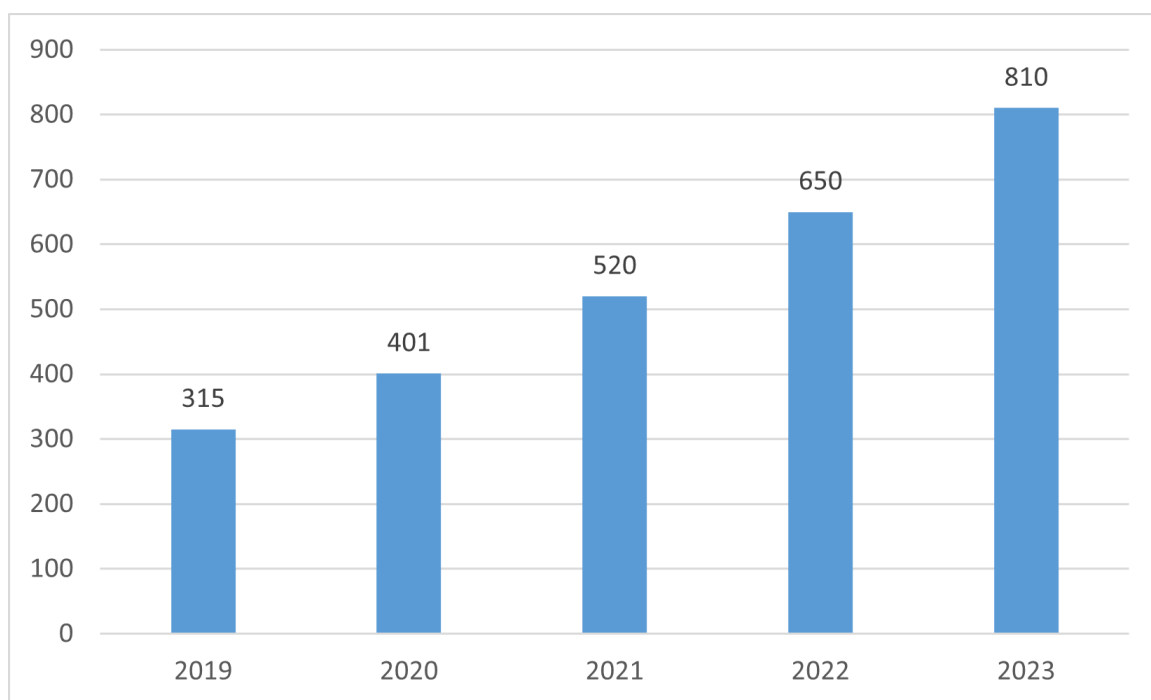


图 1.1 2019-2023 年我国数据中心机架数（万架）

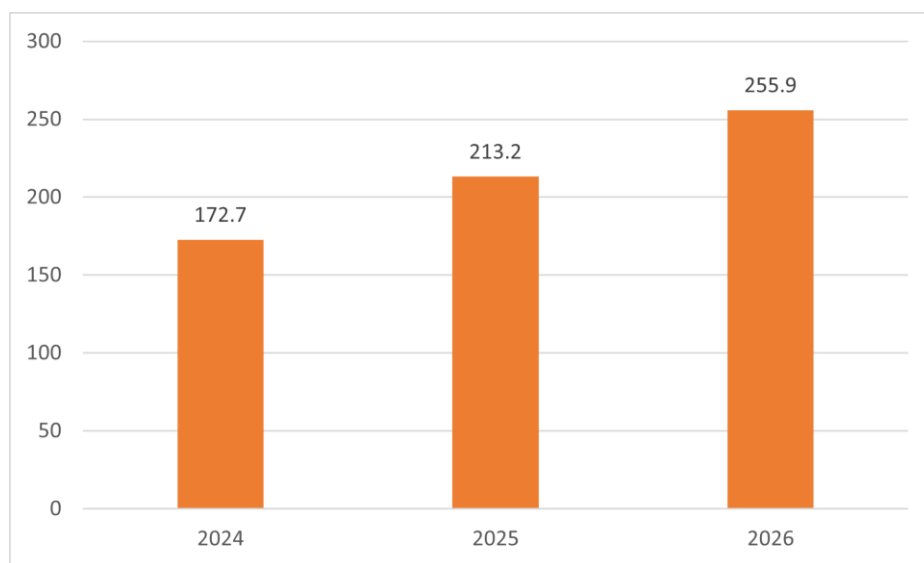


图 1.2 2024-2026 年中国分布式存储规模预测（亿元）

显然，这些蕴藏巨大价值的数据已经成为国家的战略性基础性资源。为此，国家推出一系列相关政策推动存储产业发展，先进存储技术成为重要的发展方向，例如 2023 年工业和信息化部等六部门联合印发的《算力基础设施高质量发展行动计划》^[8]指出，到 2025 年要实现存储总量超过 1800EB，其中先进存储容量占比超过 30% 的目标。

这些海量的数据给数据存储系统带来的挑战除了如何高效存储之外，还有如何保证数据的可靠性和安全性。因此，学术界和工业界对研究同时具备高可靠性、高效数据存储管理性能特性的存储系统持有浓厚兴趣。

1.2 存储技术方案

在存储领域中，集中式存储以及分布式存储^[9-11]是两种常见的存储技术方案。前一种技术方案的原理是将数据存放在单一的中心节点中，通常要求节点中的主机需具备高性能，但这往往带来高昂的经济成本，在巨量的存储需求面前，其成本更加难以承担。除开成本问题，单个中心节点还有一个先天的缺陷：当中心节点发生故障，整个系统将陷入不可用，这无疑是无法接受的。此外，系统整体性能依赖于中心节点性能^[12]，相对有限的横向扩展能力以及共享性能^[13]进一步限制了其使用场景。

考虑到前者技术上的多个不足，谷歌提出新的存储技术方案：分布式存储技术。该方案的存储原理是：数据是被分散存放到多个节点的，因此，系统具备更强的容灾能力。节点与节点之间相互独立的关系带来了扩展能力的提升，增加存储节点数量不仅对存储系统起到扩容作用，另一方面，对整个系统的 I/O 性能也有正向的增益。相比集中式存储系统中的大型主机，分布式存储系中的节点价格要低廉得多，这进一步放大了分布式存储技术的优势。诚然分布式存储系统有着多方面优势，但随着数据量

增加，存储系统中硬盘数量因此水涨船高，其发生故障的现象也变得更为常见^[14]。调查统计显示，一个文件系统在达到 Petabyte 级别时，在每一天硬盘发生故障的概率为 100%^[15]。从存储节点的角度看，几乎每天失效的数量占比能达到 1%到 2%^[16]，如 Facebook 数据中心中，一个拥有 3000 个存储节点的集群，每日最多的时候有 110 个存储节点失效从而需要进行数据修复^[17]。因此，提高分布式存储系统的容错能力已成为研究关注的焦点。

1.3 分布式存储系统

分布式存储系统（Distributed Storage System, DSS）因其具备支持高并发、强扩展性以及高可靠性的性质，在云计算服务、工业物联网、互联网等多个领域中得到广泛的应用。含有多个存储节点的集群组成了分布式存储系统，存储节点由服务器或者磁盘构成，数据被存放在这些节点中，并经由网络实现节点之间的通信以及数据管理。接下来对谷歌文件系统和 Hadoop 分布式存储系统进行介绍。

1.3.1 谷歌文件系统

为了更高效地在海量数据中搜索到目标数据以及对大文件进行读写操作，Google 设计了闭源的分布式文件系统（Google File System, GFS）。在 GFS 中，采用路径名对数据进行标识，并将数据通过目录分层组织。和常规的文件系统相同，GFS 支持对文件进行增删改查等操作，在此基础上，快照以及记录追加是谷歌文件系统特色功能，前者使系统能够便捷地进行目录树和文件的复制操作，后者使得多个用户能够在同一个时间向同一个文件中写入数据。

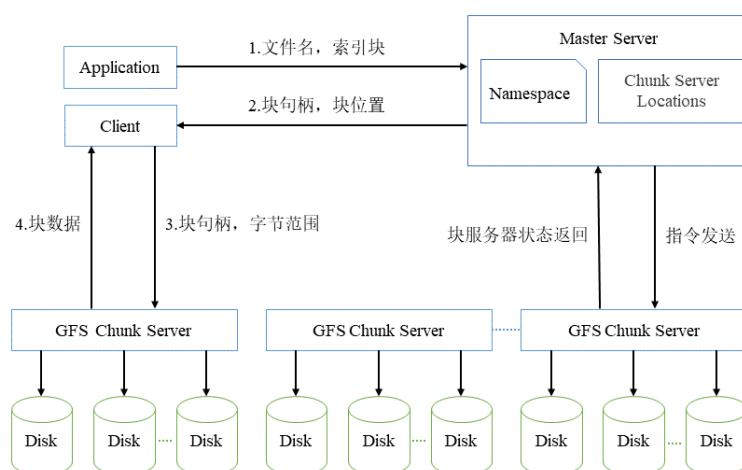


图 1.3 GFS 文件读取原理示意图

图 1.3 中展示了 GFS 集群中文件读取的过程原理。从图中可以看出，GFS 中数据

通过复制 3 个副本提升可靠性，整个系统集群包括一台实现整体管理的主服务器（Master Server）以及多台用于存储数据的块服务器（Chunk Server）组成，这些服务器能够被多个客户端（Client）访问。一般情况下，系统会将文件划分成单个大小 64MB 的数据块，并由主服务器打上一个独一无二且不可改变的标识，之后这些数据块被传输到节点中存放。主服务器负责系统中命名空间、文件-块映射表、当前数据块位置以及访问控制信息的管理，数据块以 Linux 文件的格式被块服务器存放在本地磁盘中，块服务器负责对标识以及字节范围指定的数据块内容进行读写操作。客户端需要读取 GFS 中的文件时，首先计算得到文件内数据块索引，接着发起一个文件读取的请求到主服务器，请求里面需要包括文件的名称以及数据块的索引。收到请求的主服务器将数据块句柄以及所在块服务器地址信息发送给客户端。客户端根据收到的信息向对应的块服务器发送文件读取请求，最后块服务器将文件数据发送到客户端，至此，客户端完成对文件数据的读取操作。

1.3.2 Hadoop 分布式存储系统

Hadoop 分布式存储系统（Hadoop Distributed File System, HDFS）是由 Apache 开发的一个使用分布式存储以及海量数据处理的开源框架，其对 TB 级别或者 PB 级别的数据有着强大的支撑能力。HDFS 的优势在于能够实现高吞吐量、高容错性、强扩展性以及大文件存储管理的支持。

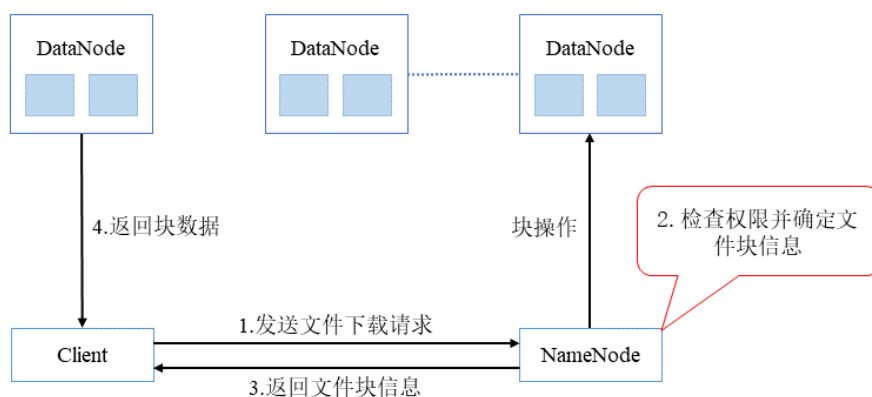


图 1.4 HDFS 文件读取原理示意图

同 GFS 一样，HDFS 也采用主从架构，在 HDFS 集群中，包含一个负责对文件系统中文件名称进行管理以及对客户端（Client）访问文件实行控制的主节点（NameNode）、多个用于存储文件数据块（单个默认大小 64MB）的从节点（DataNode）以及客户端构成。图 1.4 展示了 HDFS 中文件读取原理，与谷歌文件系统相似，客户端需要先发起请求，经过主节点（主服务器）进行客户端的读取权限核实确认之后，再将文件数据块信息返回给客户端。由接收到的数据块信息，客户端与存储数据块的从

节点建立连接，读取目标文件数据。

1.4 分布式存储系统中的容错技术

在原始的数据中添加冗余数据^[18]是为了让数据能够同时兼具高可靠性以及高可用性。多副本策略或者纠删码策略是两种常用的用于达成数据的冗余存储目的的技术方案。这两种策略的使用，使得即使系统中有某些节点发生故障，其他正常的节点依旧能够为系统正常运转提供保障。

1.4.1 多副本策略

多副本策略，即复制策略。其实现原理非常基本且直观，具体地，拷贝多份数据，并将其在集群中分散存储，以此实现容错目的。通过这种策略，当集群中有节点发生故障导致数据失效时，只要仍有节点幸存，都能够从幸存节点拷贝下载副本数据，完成原始数据的修复。下载数据的时候采用就近原则以获得较快的恢复速度。可以看出，只有所有的存放数据副本的节点在同一时间发生故障，才会导致原始数据无法修复的情形出现。使用多副本策略的存储系统需要通过增加副本数量的方式来确保存储系统的容错能力获得提高。复制策略因为只是对原始数据块进行复制，并不涉及编解码运算，因此具有操作简单，易于实现与维护、数据恢复速度快的特点。然而，正因如此，多副本策略在对数据恢复时产生的大量读写数据的操作会给节点之间的网络带来巨大的压力负载。同时，由于在复制策略中每个备份都保留着原文件所有数据信息，因此当系统存储的副本数量为 n 时，需要的存储空间是源数据量的 n 倍。即存储效率仅为 $1/n$ ，在海量数据的前提下，将带来巨大的经济成本。

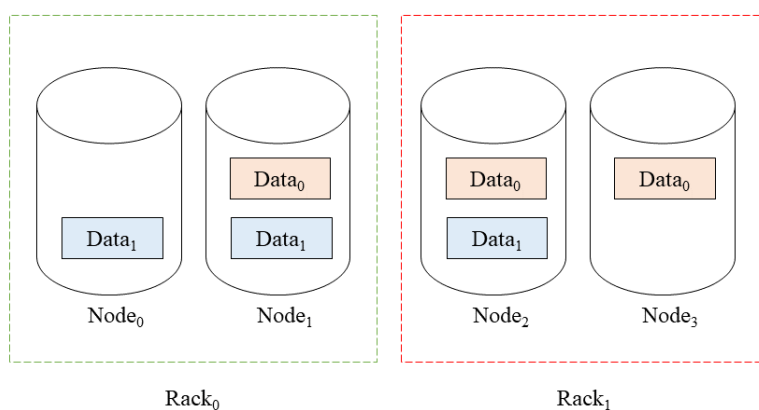


图 1.5 3 备份谷歌文件系统原理示意图

图 1.5 展示的是拥有 3 个备份数据的谷歌文件系统 GFS 原理图，数据在系统中存放的位置是均匀分散的。若单个机架内部有节点发生故障，根据就近原则，系统将在

机架内部的幸存节点下载备份数据以完成修复。只有当机架内部所有存放备份数据的节点都发生故障，系统才会选择从其他存放备份数据的机架中获取数据完成修复。在立即修复以及延迟修复两种方式中，前者以增加资源消耗为代价保证系统中备份数据的一致性。后一种修复方式的思想则与前者相反，适用于资源有限且对数据一致性要求不高的场景。

1.4.2 纠删码策略

在拥有相同容错能力的前提下，纠删码策略比多副本策略需要的存储开销更小，因而存储硬件成本更低。一开始，纠删码在信道编码^[19]领域使用，得益于其强大的前向纠错能力，随后在存储领域发挥重要作用，即恢复因磁盘损坏等故障而丢失的数据从而保证存储系统可靠性。

接下来介绍纠删码相关的主要专业名词概念：

(1) 节点 (Node)：用于存储数据块以及校验块的存储节点，可以是服务器或磁盘等。节点可以专门存储数据块或校验块，也可将两者共同存储。

(2) 数据块 (Data Block)：由原始文件等量划分得到，每一份即为一个数据块，是纠删码进行编解码操作的基本单元。

(3) 校验块 (Parity Block)：也称冗余块，是由数据块经过既定算法生成获得。

(4) 条带 (Stripe)：计算获得校验块后，数据块与其组成条带，作为纠删码技术最基础研究对象，条带间操作相互独立。

(5) 编码 (Encoding)：原始数据块经过计算获得校验块的过程。

(6) 解码 (Decoding)：由幸存数据块计算恢复失效节点中数据块的过程。

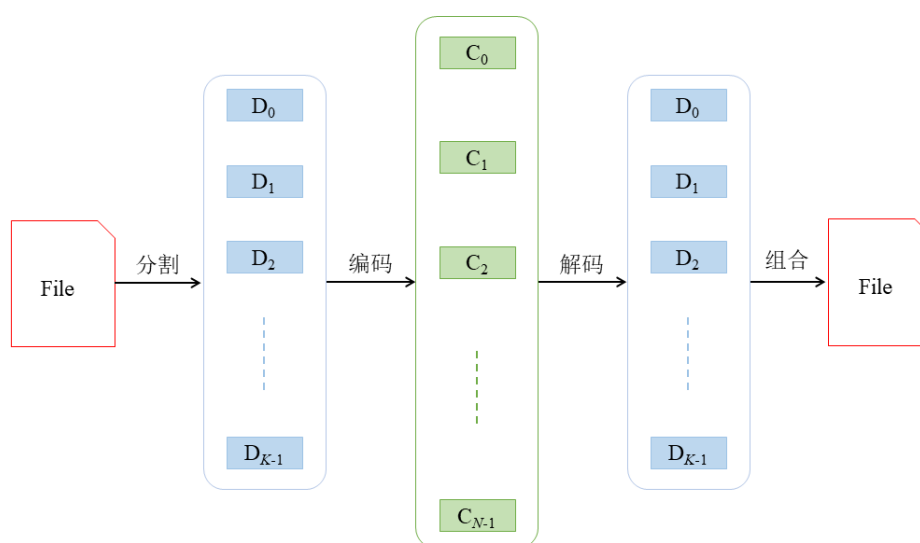


图 1.6 纠删码原理示意图

图 1.6 展示了纠删码的原理, 需要存储的原始文件被划分成 K 个大小相等的数据块 $D_0, D_1, D_2, \dots, D_{K-1}$, 再按照一定的规则编码计算得到 N 个编码块, 即 $C_0, C_1, C_2, \dots, C_{N-1}$, 将这些编码块分散存放到不同的存储节点中。当节点中有编码块发生失效需要完全恢复(解码)时, 此时要求系统的容错能力不小于当前失效编码块数量。在幸存的编码块中选取任意的 $K' \geq K$ 个, 经过解码运算, 实现原始文件数据恢复。纠删码是否为最大距离可分码(Maximum Distance Separable Codes, MDS Codes)根据是否满足条件 $K' = K$ 定义。若该纠删码为 MDS 码, 那么其具有 MDS 特性, 即 K 个数据块, 经过编码后获得包括 T 个校验块在内共计 N 个编码块, 其中任意的 K 个编码块都能够通过解码运算求得原始文件数据。换句话说, 只要失效编码块数量小于等于 T , 系统都能保证所有失效数据可恢复。

1.5 国内外纠删码研究现状

随着海量数据的快速增长, 纠删码策略在相同存储成本的条件下拥有更强大数据容错能力的优势更加突出, 因此越来越受到存储业界的欢迎及追捧。随着国内外学者对纠删码投入越来越多的关注和精力, 关于纠删码的成果日益增多。图 1.7 展示了常见的纠删码类型。下面介绍关于各类型纠删码及其编解码方法、容错能力等要点的研究现状。

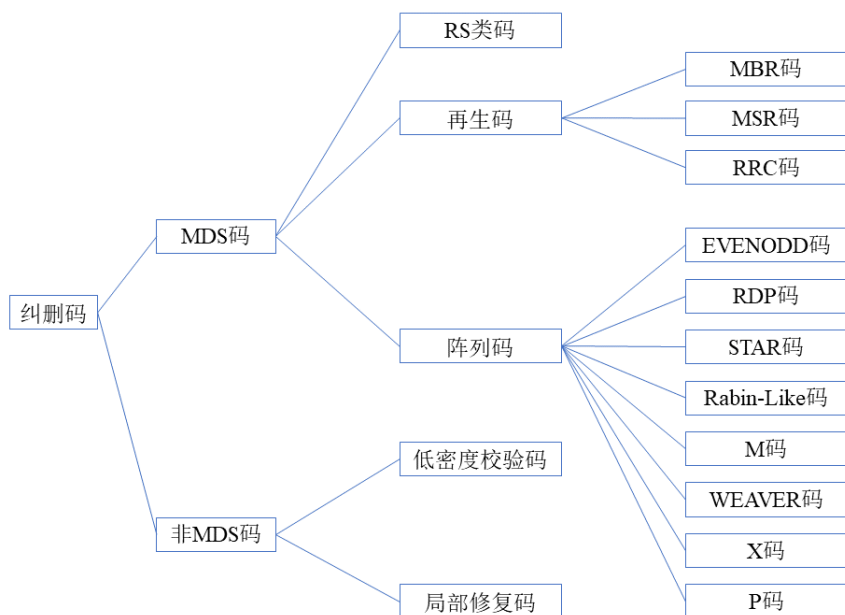


图 1.7 常见纠删码类型

1.5.1 Reed-Solomon 码

里德-所罗门码 (Reed-Solomon Codes, RS Codes)^[20]是一种经典的前向擦除控制码, 其最初被应用在信道编码领域。RS 码具备 MDS 特性, 并且支持任意码长 (需在有限域大小范围内)、码率以及容错能力, 因此在存储系统中被广泛应用, 如独立磁盘冗余阵列 (Redundant Array of Independent Disks, RAID)^[21]、Swift^[22]、Ceph^[23]等。通常 RS 码在有限域上进行编解码运算, 在有限域上, 加法操作等同于异或操作, 而乘法操作等同于多项式运算, 其中乘法运算是一个限制 RS 码性能的瓶颈点^[24, 25]。因此当前针对 RS 码的研究重点在于降低编解码过程的计算复杂度以及提升编解码效率。唐聃等人^[26]为降低解码复杂度, 设计了一种算法让幸存码元线性组合得到失效码元, 其避免了矩阵求逆操作, 并运用了译码变换矩阵的简单变换以及更低计算复杂度的乘法、加法。优化的 Berlekamp-Massey 算法^[27]通过优化迭代解码过程中的计算步骤, 有效降低时间复杂度, 提升了解码速度。Lin 等人^[28, 29]提出了具有低计算复杂度的 RS 码编解码算法, 该算法基于快速傅里叶变换 (Fast Fourier Transform, FFT)。针对以范德蒙矩阵作为校验矩阵的 RS 码, Yu 等人^[30]提出了一种基于 Reed-Muller 变换的快速编解码算法, 该算法支持校验数为 4 到 7 的 RS 码, 在此基础上改进了算法^[31], 将支持的校验数量拓展到任意值, 通过减少乘法操作进一步降低了编码过程计算复杂度。Deng 等人^[32]在大小为 256 的有限域上构造了校验长度为 3 到 5 的 RS 码, 其编解码算法基于改进的 RM 变换, 编解码过程仅需异或及循环移位操作, 兼具低计算复杂度和低空间复杂度特点。孙怡宁等人^[33]设计了提前输出算法, 该算法基于 Chase 算法, 在解码时仅需生成部分测试码字, 而不需要生成所有的测试码字, 以低时间复杂度达到与原算法误帧率性能相近水平。Tang 等人^[34]提出一种基于模块化方法的解码算法, 具有最优渐近计算复杂度 $O(n \log(n-k) + (n-k) \log_2(n-k))$ 和最小常数因子。对基于 Cauchy 矩阵的 RS 码, Blomer^[35]将有限域元素使用二元域的位矩阵表示, 通过把有限域乘法转换成异或运算的方式使编码复杂度得以降低。Tang 等人^[36]通过对计算中间结果进行复用, 从而降低编解码过程复杂度。Liu 等人^[37]提出了一种适用于任意 (N, K) RS 码的线性修复方案, 降低了修复时的 I/O 开销。

1.5.2 再生码

为降低节点修复带宽, Dimakis 等人^[38]提出了再生码 (Regenerating Codes, RGC), 并绘制了存储开销和修复带宽两者的权衡关系曲线。在再生码中, 帮助节点与再生节点都能够执行编码操作, 帮助节点先在内部对参与修复的数据节点上的数据进行线性组合, 之后将结果发送给再生节点, 再生节点对结果再次编码以此完成失效节点修复。再生码实现了有效降低修复失效节点时所需消耗带宽的目的。再生码主要分为通过最小存储开销完成对失效节点修复的最小存储再生码 (Minimum Storage Regenerating,

MSR)、利用最小化修复过程占用带宽完成失效节点修复的最小带宽再生码 (Minimum Bandwidth Regenerating, MBR) 两类。Hou 等人^[39]根据数据中心架构的特点, 对节点修复过程中数据的传输路径进行优化, 提出能够带来更低跨机架修复带宽的机架感知再生码 (Rack-aware Regenerating Codes, RRC)。之后提出^[40]了能够实现单节点修复和单机架修复最优的广义机架感知再生码 (Generalized Rack-aware Regenerating Codes, GRRC)。再生码以额外的存储开销为代价, 换取分布式存储系统在修复失效节点时占用带宽的降低, 拓宽了纠删码容错技术的研究方向。

1.5.3 阵列码

Blaum 等人^[41]提出的阵列码 (Array Codes) 是一种线性纠删码, 区别于其他纠删码, 阵列码在系统中以二维的结构形式进行组织, 即码的结构体现为一个 M 行 N 列的阵列。阵列码是一种具有低计算复杂度特点的 MDS 码, 原因在于仅通过执行循环移位以及异或 (XOR) 操作便可实现对数据的编解码, 得益于计算效率高和容易实现等优势, 阵列码在需要对大量数据进行处理分布式存储系统中得到广泛的应用。根据阵列码的编解码结构特点, 即原始文件数据与校验数据存放的位置关系, 阵列码可被分为横式阵列码以及纵式阵列码两种类型。

在横式阵列码中, 原始文件数据以及校验数据在节点中的存储位置是分开的, 即单个数据节点只存放两者的其中之一, 这种分离存储的特点带来扩展性强的优势。从容错能力角度看, 常见的横式阵列码仅能容忍 2 个或者 3 个节点失效。典型的横式阵列码有 EVENODD 码^[42]以及 RDP 码^[43], 阵列码领域在近年来吸引着学术界越来越多的关注, 在探索过程中积累了一系列重要突破。Blaum 等人^[44, 45]对广义 EVENODD 码具备 MDS 属性时参数选择进行了讨论。Huang 等人^[46]设计的 EVENODD 码可容忍 3 个失效。RTP 码由 Goel 等人^[47]通过改进 RDP 码得到, 其容错能力由 2 扩展至 3。Jiang 等人^[48]提出了可以容忍 3 个节点失效的拓展 EVENODD 码。Hou 等人^[49]参考了 RDP 码和 EVENODD 码的构造, 设计了三容错码。Lan 等人^[50]设计了具有 4 容错能力的 EVENODD 码。为了进一步降低计算复杂度, Hou 等人^[51]设计了一种新的 EVENODD 构造方法以及对应的编解码算法, 基于此, Fu 等人^[52]提出了能够容忍 3 个节点失效的扩展 EVENODD 码和相应的低计算复杂度算法。Chen 等人^[53]设计的解码算法中, 在条带解码时使用预先设定的解码路径, 由此使得阵列码的解码速度得到明显提升。Wang 等人^[54]设计了一种解码算法。该算法适用于 3 个数据节点失效的情况, 然而失效节点相对位置会对算法的效率产生影响。STAR 码^[55]由 Huang 等人提出, 与常规的 EVENODD 码使用 0、1、2 不同, 该阵列码的斜率为 -1、0、1。Hou 等人^[56]设计了一种新的 MDS 阵列码, 该码能够支持多个节点失效的修复。Zhu 等人^[57]提出了一种构造参数更灵活的 EVENODD+码构造方法, 并实现了渐进最优的编解码、更新复杂度。在

$(p-1, p+1)$ RDP 码中, 磁盘数量需要设置为素数或者素数-1, 所有值置 0 的虚拟行(仅用于理论说明, 实际系统并不存储该数据)位于码的最底部, 图 1.8 展示的是 $(4, 6)$ RDP 码结构示意图。

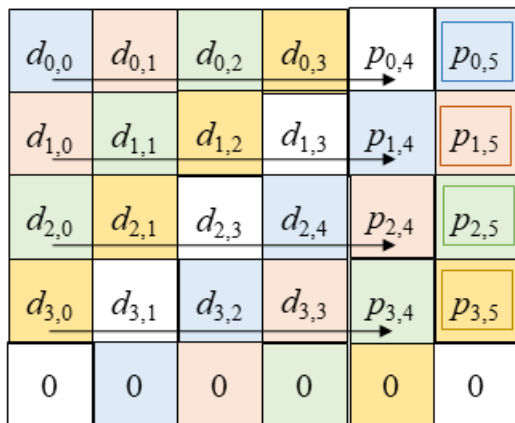


图 1.8 $(4, 6)$ RDP 码结构示意图

在纵式阵列码中, 原始文件数据与校验数据存储在相同的节点中, 因不需要像横式阵列码一样在磁盘不同存储节点频繁进行读写操作, 纵式阵列码避免了磁盘热点集中问题。原始文件数据以及校验数据在磁盘中的分布是均匀的, 由此也造成了纵式阵列码扩展性较差的劣势。常见的纵式阵列码有 Rabin-Like^[58]、M-code^[59]、WEAVER^[60]、X-Code^[61]以及 P-Code^[62]等等。图 1.9 展示了参数 $p=5$, 即 $(3, 5)$ X-Code 结构示意图。

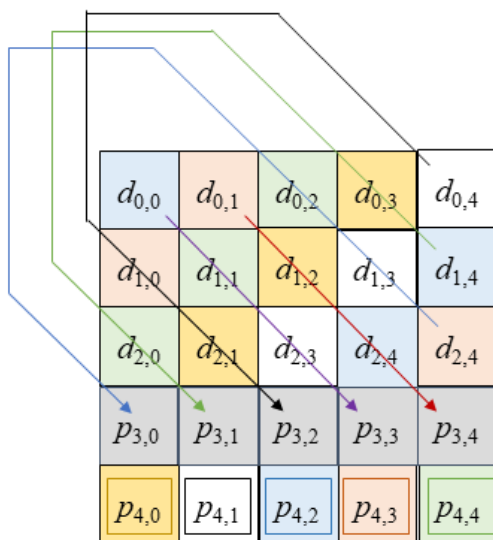


图 1.9 $(3, 5)$ X-Code 结构图

1.5.4 非 MDS 码

为了降低修复带宽，再生码选择令磁盘 I/O 开销增加的方式。而在局部修复码^[63]（Locally Repairable Codes, LRC）中，用于修复失效节点的幸存节点选择范围受到了限制，即仅可使用特定存储节点对其进行修复，这种方式降低修复节点占用的带宽，原因在于其减少了修复执行时所需数据的读取频率以及下载。局部修复码把奇偶校验块分为局部和全局两种类别，前者是同组内各数据块校验和，后者为全部数据块校验和。如图 1.10 所示，在参数(4, 3)LRC 中， D_0, D_1, D_2, D_3 为数据块， P_0, P_1 为局部校验块， P_2 为全局校验块。任意 1 个非 P_2 的数据块损坏时，可通过组内 2 个幸存节点进行修复；当仅有 P_2 这 1 个数据块失效时，需要通过 D_0, D_1, D_2, D_3 这 4 个节点完成修复；若有 2 个数据块失效，则需通过所有幸存数据块才能完成节点修复；若位于不同组内的任意 3 个数据块失效，通过其余幸存节点数据块能够完成对失效数据块的修复；当 3 个损坏的数据块位于同一个组内时，数据块将无法修复。可见，尽管该 LRC 码拥有 3 个冗余，但无法容忍任意的 3 个数据块失效。

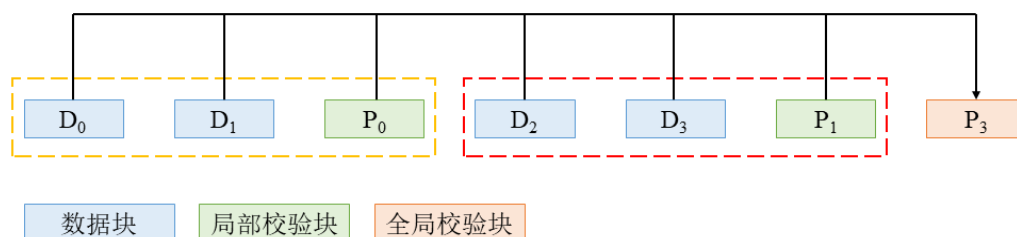


图 1.10 (4, 3)LRC 结构图

1.6 研究内容与章节安排

1.6.1 研究内容

为了减少 RS 码解码过程中所需乘法操作次数，降低解码计算复杂度，进而提升 RS 码的解码性能，本文研究了分布式存储系统中 RS 码的快速解码算法，具体如下：

(1) 提出了一种依据有限域 $GF(2^m)$ （ m 为偶数）上分圆陪集生成的范德蒙矩阵，并以该范德蒙矩阵作为校验矩阵设计了一种新的 RS 码结构。在此基础上，本文提出了一种针对有限域 $GF(2^m)$ （ m 为偶数）上 $T \times N$ 范德蒙矩阵分解方法，通过该分解方法有效减少了 RS 码解码过程中所需的乘法操作次数，降低了计算复杂度。在码字长度 $N=16$ ，校验长度 $T=4$ 和 $T=7$ 时，相比 RM 算法，本文提出的算法在解码过程中需要的乘法次数分别减少 7%和 29%。

(2) 本文参考云存储中的中继思想，根据纠删码相关理论，通过添加 1 个代理节

点 ProxyNode 对常规 HDFS 进行改进,之后将本文提出的 RS 码搭载到系统中。在改进的 HDFS 上测试了本文提出 RS 码算法在不同参数条件下的性能表现。从实验结果看出,相比 RM 算法、ISA-L 和 Jerasure2.0,在码长 $N=256$,校验长度 $T=16, 32, 64, 128$ 时,本文提出算法解码速度优势随着 T 的增加而增加,其分别提升了 5%~17%、68%~271%和 504%~976%。此外,当 $N=256, T=32$,数据块大小 PacketSize 由 128Byte 增加至 8192Byte 时,本文提出的算法编码速度由 1138MB/s 提升至 1446MB/s,提升了 21%,解码速度由 1113MB/s 提升至 1371MB/s,提升了 23%。

1.6.2 章节安排

正文内容共 5 章,结构如下:

第一章:绪论。首先对本文研究内容的研究背景和意义作了介绍。接着介绍两种常见分布式存储系统。再者阐述容错技术以及国内外纠删码相关研究现状。最后介绍本文的研究内容和章节安排。

第二章:Reed-Solomon 码。在本章中对 Reed-Solomon 码进行更深入的介绍。首先介绍了有限域的相关理论知识,其次介绍了基于生成矩阵和基于校验矩阵两种 RS 码构造方法。

第三章:Reed-Solomon 码快速解码算法。在本章中,首先介绍了多点求值和循环归一化卷积的相关概念,接着提出一种有限域 $GF(2^m)$ (m 为偶数)上分圆陪集构成范德蒙矩阵的方法,并以该范德蒙矩阵作为校验矩阵构造了一种新的 RS 码。之后提出 $T \times N$ 范德蒙矩阵分解方法并给出了详细证明。基于此矩阵分解方法,提出了减少解码过程所需乘法次数的 RS 码快速解码算法。最后分析了本文提出算法的计算复杂度,并与 RM 算法比较了有限域 $GF(2^4)$ 上的 RS 码解码过程所需乘法次数,结果表明,当码长 $N=16$,校验长度 $T=4$ 和 $T=7$ 时,本文提出的算法在解码过程中需要的乘法次数分别减少 7%和 29%。

第四章:HDFS 系统实现。本章首先介绍了实验平台(改进的 HDFS)的组成,之后对平台实现文件上传、下载以及文件的编解码的原理过程做了详细描述。最后实验测试了不同参数下包括本文提出算法在内的多种算法性能表现,并做了评估分析。结果显示,相比 RM 算法、ISA-L 和 Jerasure2.0,在码长 $N=256$,校验长度 $T=16, 32, 64, 128$ 时,本文提出的算法解码速度优势随着 T 的增加而增加,其分别提升了 5%~17%、68%~271%和 504%~976%。当 $N=256, T=32$,数据块大小 PacketSize 由 128Byte 增加至 8192Byte 时,本文提出算法编码速度和解码速度分别提升了 21%和 23%。

第五章:总结和展望。本章总结全文内容,并分析了其中仍需要改进的地方,展望了未来可能的研究方向。

第二章 Reed-Solomon 码概述

2.1 有限域

2.1.1 有限域的定义及构造

有限域^[64-66]又称伽罗华域，由法国数学家 Galois 发现。给定一个含有 n 个元素的集合 $C = \{\alpha_0, \alpha_1, \alpha_2, \dots, \alpha_{n-1}\}$ ，用“+”、“ $\times$ ”分别表示集合 C 上的加法操作和乘法操作，若集合 C 满足以下运算性质：

(1) 封闭性：对任意 $a, b \in C$ ，有

$$a + b = c, c \in C,$$

$$a \times b = d, d \in C.$$

(2) 交换律：对任意 $a, b \in C$ ，有

$$a + b = b + a,$$

$$a \times b = b \times a.$$

(3) 结合律：对任意 $a, b \in C$ ，有

$$(a + b) + c = a + (b + c),$$

$$(a \times b) \times c = a \times (b \times c).$$

(4) 加法分配律：对任意 $a, b \in C$ ，有

$$a \times (b + c) = a \times b + a \times c.$$

(5) 加法单位元：对任意 $a \in C$ ，有

$$a + 0 = a.$$

(6) 加法逆元：对任意 $a, -a \in C$ ，有

$$a + (-a) = 0.$$

(7) 乘法单位元：对任意 $a, e \in C$ ，有

$$a \times e = a.$$

(8) 乘法逆元：对任意 $a, a^{-1}, e \in C$ ，其中 e 为乘法单位元，有

$$a \times a^{-1} = e.$$

则称集合 C 为有限域，记为 $GF(n)$ ，其中集合元素个数 n 称为有限域的阶， n 满足 $n = q^m$ ，且 q 是一个素数。注意，除非做了特殊说明，否则在本文中，取 $q = 2$ 。当 $m \neq 1$ 时，有限域 $GF(2^m)$ 称为二元扩展有限域。

接下来介绍与有限域构造相关的几个基本概念：

(1) 不可约多项式： $F(x)$ 是有限域 $GF(q^m)$ 上的一个 m 次多项式，且 $GF(q^m)$ 上任意一个次数为 0 到 m 的多项式都无法满足整除 $F(x)$ 的条件，那么该多项式 $F(x)$ 在有限域 $GF(q^m)$ 上是不可约的，即不可约多项式。

(2) 本原多项式：有限域上的 m 次不可约多项式 $F(x)$ 可以整除 $x^n + 1$ ， $n = q^m - 1$ 时，则 $F(x)$ 为有限域 $GF(q^m)$ 的本原多项式。例如，令 8 次多项式 $F(x) = x^8 + x^4 + x^3 + x^2 + 1$ 且为不可约多项式， $F(x)$ 能够整除 $x^{255} + 1$ ，但无法整除多项式 $x^n + 1$ ， $(0 < n < 255)$ ，因此， $F(x)$ 是有限域 $GF(2^8)$ 上的一个本原多项式。

(3) 有限域元素的阶：令 α 是有限域 $GF(q^m)$ 上的一个非零元素，若 $\alpha^n = 1$ ，称 n 是有限域元素 α 的阶。

(4) 本原元：令 α 是有限域 $GF(q^m)$ 上的一个元素，若 $\alpha^n = 1$ ， $n = q^m - 1$ ，则元素 α 为有限域 $GF(q^m)$ 的本原元。

(5) 令 α 为有限域 $GF(2^m)$ 的本原元， $F(x)$ 是有限域上的一个本原多项式，那么有限域 $GF(2^m)$ 上的元素可以通过 α 的幂次进行表示：

$$\{0, \alpha^0, \alpha^1, \alpha^2, \dots, \alpha^{2^m-1}\}.$$

有限域上的元素除了通过幂次进行表示之外，还可以表示为多项式以及向量的形式，其表示结果与构造有限域使用的本原多项式有关。表 2.1 列出了使用本原多项式 $F(x) = x^4 + x + 1$ 构造的有限域 $GF(2^4)$ 上各元素的幂次表示、向量表示和多项式表示。

表 2.1 有限域 $GF(2^4)$ 上元素的表示方法 ($F(x) = x^4 + x + 1$)

幂次表示	向量表示	多项式表示
0	0000	0
α^0	1000	1
α^1	0100	α
α^2	0010	α^2
α^3	0001	α^3
α^4	1100	$1 + \alpha$
α^5	0110	$\alpha + \alpha^2$
α^6	0011	$\alpha^2 + \alpha^3$

续表 2.1 有限域 $GF(2^4)$ 上元素的表示方法 ($F(x) = x^4 + x + 1$)

幂次表示	向量表示	多项式表示
α^7	1101	$1 + \alpha + \alpha^3$
α^8	1010	$1 + \alpha^2$
α^9	0101	$\alpha + \alpha^3$
α^{10}	1110	$1 + \alpha + \alpha^2$
α^{11}	0111	$\alpha + \alpha^2 + \alpha^3$
α^{12}	1111	$1 + \alpha + \alpha^2 + \alpha^3$
α^{13}	1011	$1 + \alpha^2 + \alpha^3$
α^{14}	1001	$1 + \alpha^3$

2.1.2 有限域的计算

接下来以由本原多项式 $F(x) = x^4 + x + 1$ 构造的有限域 $GF(2^4)$ 为例, 说明有限域上的加减乘除运算规则, 令 $a = \alpha^5$, $b = \alpha^8$, 则

(1) 加、减法运算, 在二元扩域中, 元素间的加法和减法是相同的, 因此,

$$\begin{aligned}
 a + b &= \alpha^5 + \alpha^8 \\
 &= (\alpha + \alpha^2) + (1 + \alpha^2) \\
 &= 1 + \alpha \\
 &= \alpha^4, \\
 a - b &= \alpha^5 - \alpha^8 \\
 &= \alpha^5 + \alpha^8 \\
 &= \alpha^4.
 \end{aligned}$$

(2) 乘法运算:

$$\begin{aligned}
 a \times b &= \alpha^5 \times \alpha^8 \\
 &= \alpha^{13}.
 \end{aligned}$$

(3) 除法运算, 有限域上, 除法等同于与除数的逆元相乘, 因此

$$\begin{aligned}
 a \div b &= \alpha^5 \times (\alpha^8)^{-1} \\
 &= \alpha^{-3} \\
 &= \alpha^{12}.
 \end{aligned}$$

2.2 Reed-Solomon 码构造

2.2.1 基于生成矩阵的 Reed-Solomon 码构造

基于多项式构造 RS 码, 即基于生成矩阵进行构造^[20]。具体地, 假设由有限域 $GF(2^m)$ 上的元素构成含有 K 个数据符号构成的集合 $\{d_0, d_1, \dots, d_{K-1}\}$, 使用这些符号可以构成多项式

$$P(x) = d_0 + d_1x + \dots + d_{K-1}x^{K-1}. \quad (2.1)$$

通过求多项式 $P(x)$ 在有限域 $GF(2^m)$ 上 2^m 个不同点处的取值得到 (N, K) RS 码, 即

$$\begin{aligned} \mathbf{c} &= [c_0 \ c_1 \ \dots \ c_{2^m-1}] \\ &= [P(0) \ P(\alpha) \ \dots \ P(\alpha^{2^m-1})]. \end{aligned} \quad (2.2)$$

在 (N, K) RS 码中, 码字来自有限域 K 个信息符号编码计算结果, 这些码字构成的集合共有 $(2^m)^K$ 个元素, 原因在于单个信息符号具体取值有 2^m 种可能。一个纠错码是否为线性码, 可根据其是否符合线性运算封闭性判断, 即将任意两个码字进行线性组合, 结果仍然是该码中的合法码字。根据公式(2.2), 任选两个多项式, 若其次数不大于 $K-1$, 将两者相加得到的多项式, 其次数必定仍满足不大于 $K-1$ 。因此, RS 码符合线性码的条件。

因为 RS 码的码字可以看成在有限域 $GF(2^m)$ 上的一个 K 维向量空间, 所以将 RS 码中信息符号的个数 K 称为该码的维数。每个 RS 码的码字最多能够包含 2^m 个符号, 因此最大码长 $N = 2^m$ 。根据公式(2.1)、公式(2.2), 可将参数为 (N, K) 的 RS 码写成一个含有 K 个变量的线性方程组形式,

$$\begin{aligned} P(0) &= d_0 \\ P(\alpha) &= d_0 + d_1\alpha + \dots + d_{K-1}\alpha^{K-1} \\ P(\alpha^2) &= d_0 + d_1\alpha^2 + \dots + d_{K-1}\alpha^{2(K-1)} \\ &\vdots \\ P(\alpha^{2^m-1}) &= d_0 + d_1\alpha^{2^m-1} + \dots + d_{K-1}\alpha^{(2^m-1)(K-1)}. \end{aligned} \quad (2.3)$$

如要获得 K 个变量的值, 只需从中任选 K 个方程即可。可用矩阵相乘的形式表示公式(2.3)方程组:

$$\begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ 1 & \alpha & \alpha^2 & \dots & \alpha^{(K-1)} \\ 1 & \alpha^2 & \alpha^4 & \dots & \alpha^{2(K-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha^{2^m-1} & \alpha^{2(2^m-1)} & \dots & \alpha^{(K-1)(2^m-1)} \end{bmatrix} \times \begin{bmatrix} d_0 \\ d_1 \\ d_2 \\ \vdots \\ d_{K-1} \end{bmatrix} = \begin{bmatrix} P(0) \\ P(\alpha) \\ P(\alpha^2) \\ \vdots \\ P(\alpha^{2^m-1}) \end{bmatrix}. \quad (2.4)$$

在构造 RS 码时, 将公式(2.4)中第一个矩阵作为其生成矩阵。在生成矩阵中, 所有子方

阵都符合范德蒙矩阵定义，由于集合 $\{0, \alpha, \dots, \alpha^{2^m-1}\}$ 中不存在相等的元素，所以子方阵皆是非奇异的。在对 (N, K) RS 码解码时，任选 K 个符号进行计算，得到的 K 个符号集合 $\{d_0, d_1, \dots, d_{K-1}\}$ 是唯一的。所有的 (N, K) RS 码容错能力都为 $T = N - K$ ，换言之，RS 码符合作为 MDS 码的要求。

经过以上编码方法得到的结果码字中并未包括原信息符号，若通过设置以下条件，

$$\begin{aligned} P(0) &= d_0 \\ P(\alpha) &= d_1 \\ P(\alpha^2) &= d_2 \\ &\vdots \\ P(\alpha^{K-1}) &= d_{K-1}, \end{aligned} \quad (2.5)$$

基于 K 个多项式值，经由解码计算获取多项式 $P(x)$ 的 K 个系数，接着再进行编码。经过以上步骤获得的 RS 码即为系统 RS 码，码字中保留了原信息符号。

图 2.1 和图 2.2 以 $(8, 5)$ RS 码为例，分别展示了基于生成矩阵构成的系统 RS 码编解码原理，矩阵 $\mathbf{G}$ 表示生成矩阵。在编码过程中，生成矩阵 $\mathbf{G}$ 由一个大小为 5 行 5 列的单位矩阵以及大小为 3 行 5 列校验矩阵组合而成，之后将其与 5 个数据块执行有限域乘法运算以获得 3 个校验块。

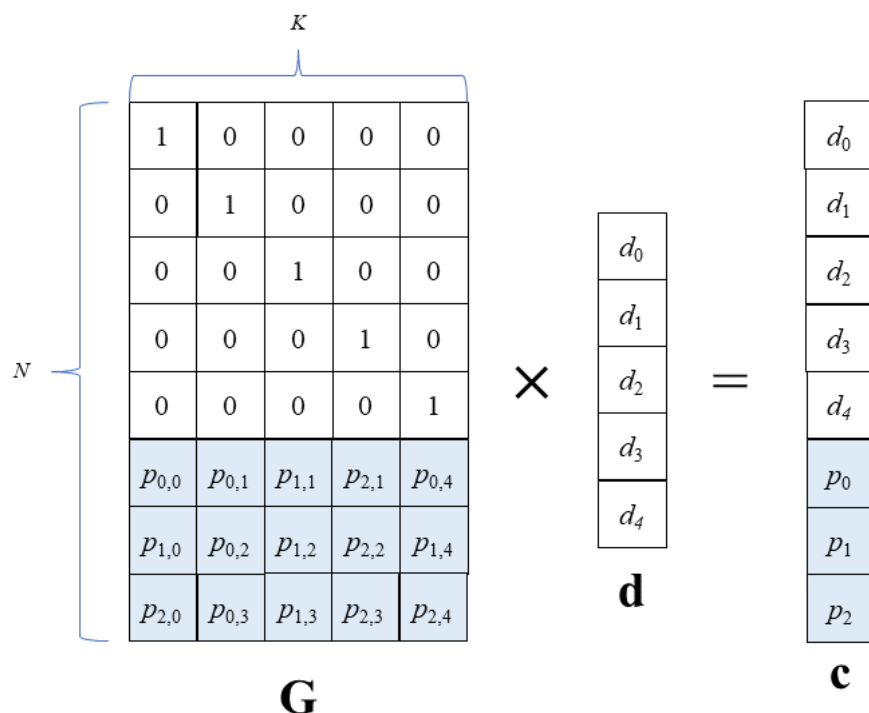


图 2.1 基于生成矩阵的系统 RS 码编码原理

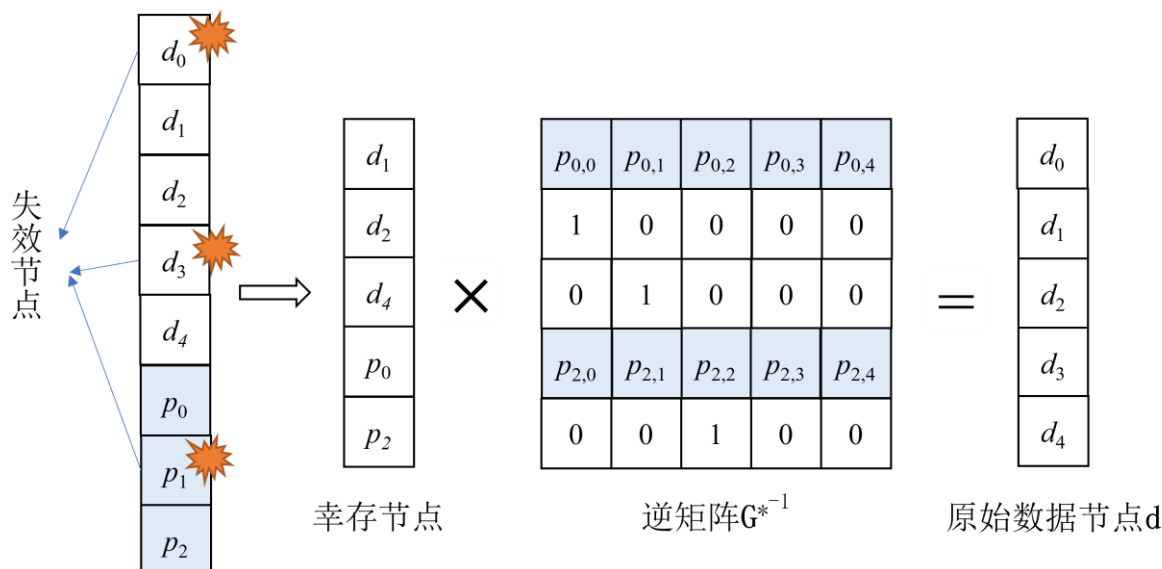


图 2.2 基于生成矩阵的系统 RS 码解码原理

在解码过程中，当数据块 d_0 、 d_3 和 p_1 失效时，首先，从生成矩阵 $\mathbf{G}$ 中选取幸存节点位置相对应的向量，构成一个新的矩阵 $\mathbf{G}^*$ ，并求出该矩阵的逆矩阵 $\mathbf{G}^{*-1}$ 。接着，由幸存节点组成向量并和逆矩阵执行有限域乘法运算，至此原始数据 $d_0, d_1, \dots, d_4$ 被成功修复。

2.2.2 基于校验矩阵的 Reed-Solomon 码构造

除了使用以上基于生成矩阵构造 RS 码之外，另一种为基于校验矩阵构造 RS 码。校验矩阵能够对码字的正确性进行判别，其满足与生成矩阵相互正交的条件。用 $\mathbf{c}$ 、 $\mathbf{H}$ 、 $\mathbf{G}$ 、 $\mathbf{0}$ 分别表示码字向量、校验矩阵、生成矩阵以及零向量，下面的等式在码字没有发生错误时恒成立，

$$\mathbf{c} \times \mathbf{H}^T = \mathbf{d} \times \mathbf{G}^T \times \mathbf{H}^T = \mathbf{0}. \quad (2.6)$$

关于校验矩阵，存在定理^[67]：码长为 N 的 RS 码 $\mathbf{c}$ ，若要使其最小码距 $d_{\min} = \kappa$ ，需要令校验矩阵 $\mathbf{H}$ 同时满足任取 $\kappa - 1$ 均构成线性无关组以及至少存在一组 κ 列呈现线性相关两个条件。

柯西矩阵和范德蒙矩阵是两种常见的 RS 码校验矩阵，接下来将对这两者进行介绍。

(1) 基于柯西矩阵的校验矩阵：

设有限域 $GF(2^m)$ 上元素集合 $\{x_0, x_1, \dots, x_{T-1}\}$ 与 $\{y_0, y_1, \dots, y_{K-1}\}$ 中，对任意的 i, j ，满足 $x_i \neq y_j$ ，定义一个基于柯西矩阵的 RS 码校验矩阵 $\mathbf{H} = [\mathbf{I}_T | \mathbf{C}]$ ，其中 $\mathbf{I}_T$ 为 T 阶单位矩阵，柯西矩阵形式如下，

$$\mathbf{C} = \begin{bmatrix} \frac{1}{x_0 + y_0} & \frac{1}{x_0 + y_1} & \cdots & \frac{1}{x_0 + y_{K-1}} \\ \frac{1}{x_1 + y_0} & \frac{1}{x_1 + y_1} & \cdots & \frac{1}{x_1 + y_{K-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{x_{T-1} + y_0} & \frac{1}{x_{T-1} + y_1} & \cdots & \frac{1}{x_{T-1} + y_{K-1}} \end{bmatrix}. \quad (2.7)$$

在柯西矩阵 $\mathbf{C}$ 中, 任意的一个 T 阶子方阵秩都为 T , 从而最小码距 $T+1$ 。令长度为 T 的校验向量 $\mathbf{p}$ 与长度为 K 的信息向量 $\mathbf{d}$ 组成码字向量 $\mathbf{c} = [\mathbf{p}|\mathbf{d}]$ 。在编码过程中, 由公式

$$\mathbf{c} \times \mathbf{H}^T = \mathbf{0}. \quad (2.8)$$

通过信息向量计算得到校验向量,

$$\mathbf{0} = \mathbf{c} \times \mathbf{H}^T = [\mathbf{p}|\mathbf{d}] \times \begin{bmatrix} \mathbf{H}_{enc}^T \\ \mathbf{H}_{aux}^T \end{bmatrix}. \quad (2.9)$$

上式中 $\mathbf{H}_{enc}$ 由校验矩阵 $\mathbf{H}$ 的前 T 列组成, $\mathbf{H}_{aux}$ 由 $\mathbf{d}$ 在校验矩阵中对应位置的列向量组成, 即 $\mathbf{p} = \mathbf{d} \times \mathbf{H}_{aux}^T \times (\mathbf{H}_{enc}^T)^{-1}$ 。在解码过程中, 由公式

$$\mathbf{e} = \mathbf{r} \times \mathbf{H}^T \times (\mathbf{H}_{dec}^T)^{-1}, \quad (2.10)$$

求得擦除位置符号。其中 $\mathbf{e}$ 为擦除符号组成的向量, $\mathbf{r}$ 为将擦除位置上的码字置为零后得到的向量, $\mathbf{H}_{dec}$ 由码字擦除符号位置在校验矩阵中对应位置向量组成。在柯西矩阵中, 任意一个 T 阶子矩阵行列式不为零, 因此逆矩阵 $(\mathbf{H}_{dec}^T)^{-1}$ 必定存在, 即 RS 码最多能够对 T 个擦除符号进行修复。

(2) 基于范德蒙矩阵的校验矩阵:

有限域不同的元素构成集合 $\{\omega_0, \omega_1, \dots, \omega_{N-1}\}$, 其中 $N \leq 2^m$ 。构造一个范德蒙矩阵作为参数为 (N, K) 的 RS 码的校验矩阵, 如下所示,

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ \omega_0 & \omega_1 & \omega_2 & \cdots & \omega_{N-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \omega_0^{T-1} & \omega_1^{T-1} & \omega_2^{T-1} & \cdots & \omega_{N-1}^{T-1} \end{bmatrix}. \quad (2.11)$$

在上述的范德蒙矩阵中, 集合上不存在相等的元素, 因此任意 T 阶方阵的秩为 T , 使用该校验矩阵构成的 RS 码最小码距 $T+1$, 编解码过程与基于柯西矩阵的 RS 码编解码类似。

本文研究基于范德蒙矩阵构造校验矩阵的 RS 码解码算法, 具体码字结构以及相应的解码算法在下一章详细描述。

2.3 本章小结

在本章中，本文介绍了 Reed-Solomon 码的相关理论知识。首先介绍了有限域的定义、构造方法，并通过举例详细说明了在二元扩展有限域 $GF(2^m)$ 上元素的运算规则。其次，介绍了两种 RS 码的构造方法：基于生成矩阵，亦称基于多项式的构造方法以及基于校验矩阵的构造方法。并对两种构造方法的编解码过程进行了详细解释。

第三章 Reed-Solomon 码解码算法

在本章中，提出了一种基于范德蒙矩阵分解的 RS 码快速解码算法。该算法通过将作为校验矩阵的范德蒙矩阵分解为多个子矩阵相乘的形式，减少了解码计算过程中非需要的乘法次数，降低了计算复杂度，从而提升 RS 码的解码效率。之后将提出的 RS 码解码算法与基于 RM 变换的算法进行了解码复杂度的对比分析。

3.1 基础理论介绍

3.1.1 多点求值与归一化循环卷积

在本节中，首先对一些多点求值与归一化循环卷积的相关概念进行阐述。

在二元扩展有限域 $GF(2^m)$ 中，令 m 为偶数， α 是有限域 $GF(2^m)$ 的本原元。

(1) 分圆陪集： m_k 是满足 $2^{m_k} c_k \equiv c_k \pmod{2^m - 1}$, $0 \leq c_k \leq 2^m - 2$ 的最小正整数，则称集合 $C^{(k)} = \{c_k, 2c_k, \dots, 2^{m_k-1}c_k\}$ 为分圆陪集， c_k 为分圆陪集 $C^{(k)}$ 的生成元。

(2) 二元共轭类：有限域上元素 α^{c_k} 生成的集合 $\{\alpha^{c_k}, \alpha^{2c_k}, \alpha^{2^2c_k}, \dots, \alpha^{2^{m_k-1}c_k}\}$ 称为元素 α^{c_k} 的二元共轭类， α^{c_k} 为第 k 个二元共轭类的生成元， m_k 为第 k 个二元共轭类长度， $m_k \mid m$ ，即 m_k 是 m 的因子。

(3) 极小多项式：元素 $\mu \in GF(2^m)$ 在其子域 $GF(2^{m_j}) \subset GF(2^m)$ 上的极小多项式，是指满足条件系数取自子域 $GF(2^{m_j})$ 且 $Q(\mu) = 0$ 条件下的最低次数的首一多项式 $Q(x)$ 。

令次数为 m 的多项式 $Q_k(x)$ 表示有限域 $GF(2)$ 上元素 α^{c_k} 的极小多项式，满足 $Q_k(x) \mid x^{2^{m/2}} + x + 1$ ，即 $x^{2^{m/2}} + x + 1$ 能被 $Q_k(x)$ 整除，对任意的有限域 $GF(2^m)$ ， m 为偶数，有 $x^{2^{m/2}} + x + 1 \mid x^{2^m} + x$ ，存在二元共轭类 $\{\alpha^{c_k}, \alpha^{2c_k}, \alpha^{2^2c_k}, \dots, \alpha^{2^{m_k-1}c_k}\}$ ，其生成元满足 $\alpha^{c_k} = \alpha^{2^{m/2}c_k} + 1$ 。

(4) 摩尔矩阵：若矩阵元素构成满足：

$$\mathbf{M} = \begin{bmatrix} \alpha_1^{q^0} & \alpha_1^{q^1} & \dots & \alpha_1^{q^{t-1}} \\ \alpha_2^{q^0} & \alpha_2^{q^1} & \dots & \alpha_2^{q^{t-1}} \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_t^{q^0} & \alpha_t^{q^1} & \dots & \alpha_t^{q^{t-1}} \end{bmatrix}, \quad (3.1)$$

q 是一个素数，则称矩阵 $\mathbf{M}$ 为摩尔矩阵。若矩阵既是摩尔矩阵，同时也是范德蒙矩阵，则称该矩阵为摩尔-范德蒙矩阵。

(5) 基底循环矩阵：若矩阵的每一行都是由它的前一行通过向左（右）循环移动一个位置得到，则称其为循环矩阵。当矩阵首行是一组正规基时，称该矩阵为基底循环矩阵。设有限域 $GF(2^m)$ 上的一组正规基为 $[\eta^{2^0} \eta^{2^1} \dots \eta^{2^{m-1}}]$ ，可以得到基底循环矩

阵

$$\mathbf{C} = \begin{bmatrix} \eta^{2^0} & \eta^{2^1} & \dots & \eta^{2^{m-1}} \\ \eta^{2^1} & \eta^{2^2} & \dots & \eta^{2^0} \\ \vdots & \vdots & \ddots & \vdots \\ \eta^{2^{m-1}} & \eta^{2^0} & \dots & \eta^{2^2} \end{bmatrix}, \quad (3.2)$$

矩阵 $\mathbf{C}$ 同时也是 $q=2$ 的摩尔矩阵。

(6) m 点循环卷积: 元素 $a_i, b_i, c_i \in GF(2^m)$,

$$\begin{aligned} a(x) &= \sum_{i=0}^{m-1} a_i x^i, \\ b(x) &= \sum_{i=0}^{m-1} b_i x^i, \\ c(x) &= \sum_{i=0}^{m-1} c_i x^i. \end{aligned} \quad (3.3)$$

多项式乘法 $a(x) = b(x)c(x) \pmod{x^m - 1}$ 称为 m 点循环卷积, 并可写成矩阵相乘形式:

$$\begin{bmatrix} a_0 \\ a_{m-1} \\ \vdots \\ a_1 \end{bmatrix} = \begin{bmatrix} b_0 & b_{m-1} & \dots & b_1 \\ b_{m-1} & b_{m-2} & \dots & b_0 \\ \vdots & \vdots & \ddots & \vdots \\ b_1 & b_0 & \dots & b_2 \end{bmatrix} \times \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_{m-1} \end{bmatrix}. \quad (3.4)$$

长度为 m 的归一化循环卷积是一种特殊形式的循环卷积, 其条件为多项式 $b(x)$ 的系数结构满足 $b(x) = \eta^{2^0} + \eta^{2^{m-1}}x + \dots + \eta^{2^1}x^{m-1}$ 。

3.1.2 一种新的 Reed-Solomon 码编码结构

在通常定义的 RS 码中, 元素 ω_0 的值定义为 0, $\omega_i = \alpha^{i-1}$, $1 \leq i \leq N-1$ 。作为对照, 本文将根据分圆陪集中的元素, 对 $\{\omega_0, \omega_1, \dots, \omega_{N-1}\}$ 的顺序进行重新排列, 即使使用不同的校验矩阵构造 RS 码。

根据分圆陪集 $C^{(k)}$ 生成的元素 α^{c_k} , $1 \leq c_k \leq 2^m - 2$, 构造 $T \times m_k$ 的矩阵 $\mathbf{H}_{c_k}$, 如下所示,

$$\mathbf{H}_{c_k} = \begin{bmatrix} (\alpha^{c_k 2^0})^0 & (\alpha^{c_k 2^1})^0 & \dots & (\alpha^{c_k 2^{m_k-1}})^0 \\ (\alpha^{c_k 2^0})^1 & (\alpha^{c_k 2^1})^1 & \dots & (\alpha^{c_k 2^{m_k-1}})^1 \\ \vdots & \vdots & \ddots & \vdots \\ (\alpha^{c_k 2^0})^{T-1} & (\alpha^{c_k 2^1})^{T-1} & \dots & (\alpha^{c_k 2^{m_k-1}})^{T-1} \end{bmatrix}. \quad (3.5)$$

此外, 定义 T 行 2 列矩阵 $\mathbf{H}_0$, 如下所示,

$$\mathbf{H}_0 = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}^T. \quad (3.6)$$

接着, 由矩阵 $\mathbf{H}_0$ 和所有根据分圆陪集构造的矩阵 $\mathbf{H}_{c_k}$ 组成范德蒙矩阵 $\bar{\mathbf{H}}$,

$$\bar{\mathbf{H}} = [\mathbf{H}_0 \ \mathbf{H}_1 \ \mathbf{H}_2 \ \dots \ \mathbf{H}_{c_t}]. \quad (3.7)$$

给定长度为 N 的 RS 码码字 $\mathbf{c} = [c_0 \ c_1 \ c_2 \ \dots \ c_{N-1}]$ 和一个由矩阵 $\bar{\mathbf{H}}$ 前为 N 列构成的校验矩阵 $\mathbf{H}$, $\mathbf{H}$ 为范德蒙矩阵, 当 RS 码字中没有发生错误时, 恒有

$$\mathbf{0} = \mathbf{c} \times \mathbf{H}^T. \quad (3.8)$$

令码字 $\mathbf{c} = [\mathbf{p}|\mathbf{d}]$, 其中 $\mathbf{p}$ 是长度为 T 的校验向量, $\mathbf{d}$ 是长度为 K 的数据向量。令 $\mathbf{H} = [\mathbf{H}_{enc}|\mathbf{H}_{aux}]$, 其中 $\mathbf{H}_{enc}$ 由校验矩阵 $\mathbf{H}$ 的前 T 列组成, $\mathbf{H}_{aux}$ 由 $\mathbf{H}$ 剩下的 K 列组成, 则

$$\mathbf{0} = \mathbf{c} \times \mathbf{H}^T = [\mathbf{p}|\mathbf{d}] \times \begin{bmatrix} \mathbf{H}_{enc}^T \\ \mathbf{H}_{aux}^T \end{bmatrix}. \quad (3.9)$$

假设码字向量 $\mathbf{c}$ 中有 T 个符号失效, 即发生了擦除, 可以通过以下解码过程恢复擦除的符号。令 $\mathbf{e} = [c_{e_0} \ c_{e_1} \ \dots \ c_{e_{T-1}}]$ 为 RS 码中 T 失效符号所构成的向量, $\mathbf{r} = [r_0 \ r_1 \ \dots \ r_{N-1}]$ 表示接收到的发生 T 个符号擦除的码字向量, 其中 $r_i = 0, i \in \{e_0, e_1, \dots, e_{T-1}\}$, 否则 $r_i = c_i$ 。 T 行 T 列的可逆矩阵 $\mathbf{H}_{dec}$ 由码字向量中擦除符号位置在校验矩阵 $\mathbf{H}$ 中对应的向量构成。解码过程分为以下两步:

步骤 1: 通过公式 $\mathbf{s} = \mathbf{r} \times \mathbf{H}^T$ 计算伴随式 $\mathbf{s}$;

步骤 2: 通过公式 $\mathbf{e} \times \mathbf{H}_{dec}^T = \mathbf{s}$ 求得擦除码字向量 $\mathbf{e}$ 。

(N, K) RS 码编码可以看成是一个修复码字前 $T = N - K$ 个被擦除符号的解码特例。

3.2 范德蒙矩阵分解

设由 α^{c_k} 生成的分圆陪集中元素的个数 m 为偶数。接下来对公式(3.5)中的范德蒙矩阵 $\mathbf{H}_{c_k}$ 进行分解。令 m 为偶数, α 是有限域 $GF(2^m)$ 上的本原元。设 $\beta = \alpha^{c_k} \in GF(2^m)$, 且 β 满足 $\beta = \beta^{2^{m/2}} + 1$, β 的阶 $\text{ord}(\beta) = 2^m - 1$ 。

定理 1: 对任意的一个有限域 $GF(2^m)$, 其中 m 为偶数, 大小为 m 行 T 列的矩阵 $\mathbf{H}_{c_k}^T$ 可以被分解成多个子矩阵相乘的形式, 具体地,

$$\begin{aligned}
 \mathbf{H}_{c_k}^T &= \begin{bmatrix} (\alpha^{c_k 2^0})^0 & (\alpha^{c_k 2^0})^1 & \cdots & (\alpha^{c_k 2^0})^{T-1} \\ (\alpha^{c_k 2^1})^0 & (\alpha^{c_k 2^1})^1 & \cdots & (\alpha^{c_k 2^1})^{T-1} \\ \vdots & \vdots & \ddots & \vdots \\ (\alpha^{c_k 2^{m-1}})^0 & (\alpha^{c_k 2^{m-1}})^1 & \cdots & (\alpha^{c_k 2^{m-1}})^{T-1} \end{bmatrix} \\
 &= \left[\begin{array}{c|c} \mathbf{I}_{m/2} & \mathbf{O} \\ \hline \mathbf{I}_{m/2} & \mathbf{I}_{m/2} \end{array} \right] \times \left[\begin{array}{c|c} \begin{matrix} \beta^{2^0} & & & \\ & \beta^{2^1} & & \\ & & \beta^{2^2} & \\ & & & \ddots \\ \mathbf{O} & & & \beta^{2^{m_k/2-1}} \end{matrix} & \begin{matrix} \\ \\ \\ \\ \mathbf{I}_{m/2} \end{matrix} \\ \hline \mathbf{O} & \mathbf{I}_{m/2} \end{array} \right] \times \\
 &\quad \left[\begin{array}{c|c} \mathbf{V}_{m/2} & \mathbf{O} \\ \hline \mathbf{O} & \mathbf{V}_{m/2} \end{array} \right] \times \mathbf{\Pi} \times \mathbf{B} \times \mathbf{E}_{c_k},
 \end{aligned} \tag{3.10}$$

其中 $\mathbf{I}_{m/2}$ 表示阶数为 $m/2$ 的单位矩阵， $\mathbf{O}$ 表示元素全为零的矩阵， $\mathbf{V}_{m/2}$ 是 $m/2$ 行 $m/2$ 列的摩尔-范德蒙矩阵，矩阵 $\mathbf{\Pi}$ 为置换矩阵。矩阵 $\mathbf{E}_{c_k}$ 中元素组成将由定理 1 证明过程中的公式(3.12)给出。定理 1 的证明过程如下：

定义一个 m 行 m 列的摩尔-范德蒙矩阵 $\mathbf{V}_m$ ，

$$\mathbf{V}_m = \begin{bmatrix} (\alpha^{c_k 2^0})^0 & (\alpha^{c_k 2^0})^1 & \cdots & (\alpha^{c_k 2^0})^{m-1} \\ (\alpha^{c_k 2^1})^0 & (\alpha^{c_k 2^1})^1 & \cdots & (\alpha^{c_k 2^1})^{m-1} \\ \vdots & \vdots & \ddots & \vdots \\ (\alpha^{c_k 2^{m-1}})^0 & (\alpha^{c_k 2^{m-1}})^1 & \cdots & (\alpha^{c_k 2^{m-1}})^{m-1} \end{bmatrix}. \tag{3.11}$$

α^{c_k} 是有限域 $GF(2^m)$ 中的非零元素，且其在 $GF(2)$ 中的极小多项式次数为 m ，因此矩阵 $\mathbf{V}_m$ 中的第 0 行向量 $\mathbf{v}_0 = [(\alpha^{c_k 2^0})^0 \ (\alpha^{c_k 2^0})^1 \ \cdots \ (\alpha^{c_k 2^0})^{m-1}]$ 是有限域 $GF(2^m)$ 上的一个多项式基。那么，必定存在二元矩阵 $\mathbf{E}_{c_k}$ ，满足公式

$$[(\alpha^{c_k 2^0})^0 \ (\alpha^{c_k 2^0})^1 \ \cdots \ (\alpha^{c_k 2^0})^{T-1}] = \mathbf{v}_0 \times \mathbf{E}_{c_k}, \tag{3.12}$$

其中，向量 $[(\alpha^{c_k 2^0})^0 \ (\alpha^{c_k 2^0})^1 \ \cdots \ (\alpha^{c_k 2^0})^{T-1}]$ 是矩阵 $\mathbf{H}_{c_k}$ 的第 0 行向量。令 $e_{i,j}$ 表示二元矩阵 $\mathbf{E}_{c_k}$ 上第 i 行的第 j 列元素，向量 $\mathbf{e}_j = [e_{0,j} \ e_{1,j} \ \cdots \ e_{m-1,j}]^T$ 表示 $\mathbf{E}_{c_k}$ 中第 j 列向量， $(\alpha^{c_k 2^i})^j$ 表示 $\mathbf{H}_{c_k}^T$ 第 i 行的第 j 列元素， $i = 0, 1, \dots, m-1, j = 0, 1, \dots, T-1$ ，结合公式(3.12)，有

$$\begin{aligned}
 (\alpha^{c_k 2^0})^j &= \mathbf{v}_0 \times \mathbf{e}_j \\
 &= (\alpha^{c_k})^0 \times e_{0,j} + (\alpha^{c_k})^1 \times e_{1,j} + \cdots + (\alpha^{c_k})^{m-1} \times e_{m-1,j},
 \end{aligned} \tag{3.13}$$

$$\begin{aligned}
 (\alpha^{c_k 2^i})^j &= ((\alpha^{c_k})^j)^{2^i} = (\mathbf{v}_0 \times \mathbf{e}_j)^{2^i} \\
 &= ((\alpha^{c_k})^0 \times e_{0,j} + (\alpha^{c_k})^1 \times e_{1,j} + \dots + (\alpha^{c_k})^{m-1} \times e_{m-1,j})^{2^i} \\
 &= (\alpha^{c_k})^{2^i 0} \times e_{0,j} + (\alpha^{c_k})^{2^i 1} \times e_{1,j} + \dots + (\alpha^{c_k})^{2^i(m-1)} \times e_{m-1,j} \\
 &= \mathbf{v}_i \times \mathbf{e}_j,
 \end{aligned} \tag{3.14}$$

上式中, $\mathbf{v}_i$ 是 $\mathbf{V}_m$ 的第 i 行向量, $i = 0, 1, \dots, m-1$ 。

将矩阵 $\mathbf{V}_m$ 写成两个可逆矩阵相乘的形式, 即

$$\mathbf{V}_m = \mathbf{L} \times \mathbf{R}. \tag{3.15}$$

并令 $\mathbf{L}$ 中元素如下,

$$\begin{aligned}
 \mathbf{L} &= \begin{bmatrix} 1 & \alpha^{c_k} & 0 & 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 1 & \alpha^{2c_k} & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & \dots & 0 & 1 & \alpha^{2^{m/2-1}c_k} \\ \hline 1 & \alpha^{2^{m/2}c_k} & 0 & 0 & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 1 & \alpha^{2^{m/2+1}c_k} & 0 & \dots & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & 0 & 0 & \dots & 0 & 1 & \alpha^{2^{m-1}c_k} \end{bmatrix} \\
 &= \left[\begin{array}{c|c} \mathbf{I}_{m/2} & \mathbf{O} \\ \hline \mathbf{I}_{m/2} & \mathbf{I}_{m/2} \end{array} \right] \times \left[\begin{array}{c|c} \begin{matrix} \beta^{2^0} & & & \\ & \beta^{2^1} & & \\ & & \beta^{2^2} & \\ & & & \ddots \\ & & & & \beta^{2^{m/2-1}} \end{matrix} & \\ \hline \mathbf{O} & \mathbf{I}_{m/2} \end{array} \right] \times \mathbf{\Pi}.
 \end{aligned} \tag{3.16}$$

其中, $\mathbf{\Pi}$ 中元素的组成如下, $\mathbf{\Pi}_{i,j}$ 表示矩阵中第 i 行的第 j 列元素,

$$\mathbf{\Pi}_{i,j} = \begin{cases} 1, & i = 0, 1, \dots, m/2 - 1, j = 2i, \\ 1, & i = m/2, m/2 + 1, \dots, m - 1, \\ & j = 2i + 1 - m, \\ 0, & otherwise. \end{cases}$$

接下来证明矩阵 $\mathbf{R}$ 可写成如下形式,

$$\mathbf{R} = \left[\begin{array}{c|c} \mathbf{V}_{m/2} & \mathbf{O} \\ \hline \mathbf{O} & \mathbf{V}_{m/2} \end{array} \right] \times \mathbf{\Pi} \times \mathbf{B}. \tag{3.17}$$

上式中 $\mathbf{V}_{m/2}$ 是 $m/2$ 阶摩尔-范德蒙矩阵,

$$\mathbf{V}_{m/2} = \begin{bmatrix} (\delta^{2^0})^0 & (\delta^{2^0})^1 & \dots & (\delta^{2^0})^{m/2-1} \\ (\delta^{2^1})^0 & (\delta^{2^1})^1 & \dots & (\delta^{2^1})^{m/2-1} \\ \vdots & \vdots & \ddots & \vdots \\ (\delta^{2^{m/2-1}})^0 & (\delta^{2^{m/2-1}})^1 & \dots & (\delta^{2^{m/2-1}})^{m/2-1} \end{bmatrix}, \quad (3.18)$$

矩阵中 $\delta = (\alpha^{2^{m/2}+1})^{c_k} \in GF(2^{m/2})$ 。

引理 1: 在 $GF(2)$ 及二元扩展域中, 定义一个多项式

$$\theta(x) = \sum_{i=0}^{m-1} \theta_i x^i, \quad (3.19)$$

令多项式 $r_i x + r_0$ 表示公式(3.19)中多项式除以二次多项式 $x^2 + x + \varepsilon$, $\varepsilon \in GF(2^m)$ 得到的余数多项式, 则有

$$\begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix} = \mathbf{U} \times \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{m-1} \end{bmatrix}, \quad (3.20)$$

余数矩阵

$$\mathbf{U} = \begin{bmatrix} r_{0,0} & r_{0,1} & \dots & r_{0,m-1} \\ r_{1,0} & r_{1,1} & \dots & r_{1,m-1} \end{bmatrix}, \quad (3.21)$$

矩阵中元素

$$\begin{cases} r_{1,i+1} = r_{1,i} + r_{0,i} \\ r_{0,i+1} = \varepsilon r_{1,i} \end{cases}, \quad (3.22)$$

初始条件设为

$$\begin{cases} r_{1,0} = 0 \\ r_{0,0} = 1 \end{cases}. \quad (3.23)$$

对引理 1 的证明如下:

设 $x^i \equiv r_{1,i}x + r_{0,i} \pmod{x^2 + x + \varepsilon}$ 。由

$$\begin{aligned} r_1 x + r_0 &\equiv \sum_{i=0}^{m-1} \theta_i x^i \\ &\equiv \sum_{i=0}^{m-1} \theta_i (r_{1,i}x + r_{0,i}) \\ &\equiv \left(\sum_{i=0}^{m-1} r_{1,i} \theta_i \right) x + \sum_{i=0}^{m-1} r_{0,i} \theta_i \pmod{x^2 + x + \varepsilon}, \end{aligned} \quad (3.24)$$

可得,

$$\begin{cases} r_1 = \sum_{i=0}^{m-1} r_{1,i} \theta_i \\ r_0 = \sum_{i=0}^{m-1} r_{0,i} \theta_i \end{cases}, \quad (3.25)$$

即

$$\begin{bmatrix} r_0 \\ r_1 \end{bmatrix} = \begin{bmatrix} r_{0,0} & r_{0,1} & \cdots & r_{0,m-1} \\ r_{1,0} & r_{1,1} & \cdots & r_{1,m-1} \end{bmatrix} \times \begin{bmatrix} \theta_0 \\ \theta_1 \\ \vdots \\ \theta_{m-1} \end{bmatrix}. \quad (3.26)$$

进一步地,

$$\begin{aligned} x^{i+1} &\equiv x^i x \\ &\equiv (r_{1,i} x + r_{0,i}) x \\ &\equiv r_{1,i} x^2 + r_{0,i} x \\ &\equiv r_{1,i} (x + \varepsilon) + r_{0,i} x \\ &\equiv (r_{1,i} + r_{0,i}) x + \varepsilon r_{1,i} \\ &\equiv (r_{1,i+1} x + r_{0,i+1}) \pmod{x^2 + x + \varepsilon}. \end{aligned} \quad (3.27)$$

引理 1 证明完成。

当 $\varepsilon = \delta = (\alpha^{2^{m/2}+1})^{c_k} \in GF(2^{m/2})$ 时, 余数矩阵 $\mathbf{U} = \mathbf{U}(0)$ 上的元素都是子域 $GF(2^{m/2}) \subset GF(2^m)$ 中的元素, 记 $[\mathbf{U}]^l$ 表示将矩阵 $[\mathbf{U}]$ 中的每个元素单独取 l 次幂后得到的新矩阵, 若 $\varepsilon = \delta^{2^i}$, 则余数矩阵满足 $\mathbf{U}(i) = [\mathbf{U}(i)]^{2^i} = [\mathbf{U}]^{2^i}$ 。

设 m 为大于 2 的偶数, $\delta = (\alpha^{2^{m/2}+1})^{c_k} \in GF(2^{m/2})$, 则子域 $GF(2^{m/2}) \subset GF(2^m)$ 上的一组多项式基为 $[\delta^0 \ \delta^1 \ \dots \ \delta^{m/2-1}]$ 。

结合余数矩阵 $\mathbf{U}$ 的定义, 并设 $\varepsilon = \delta = (\alpha^{2^{m/2}+1})^{c_k} \in GF(2^{m/2})$, m 为大于 2 的偶数, 则存在一个可逆矩阵 $\mathbf{B}$ 使得以下等式成立,

$$\mathbf{U} = \left[\begin{array}{c|c|c|c|c|c|c|c|c} \delta^0 & 0 & \delta^1 & 0 & \delta^2 & 0 & \dots & \delta^{m/2-1} & 0 \\ \hline 0 & \delta^0 & 0 & \delta^1 & 0 & \delta^2 & \dots & 0 & \delta^{m/2-1} \end{array} \right] \times \mathbf{B}. \quad (3.28)$$

设定一个多项式

$$\theta(x) = \sum_{i=0}^{m-1} \theta_i x^i, \quad (3.29)$$

令其除以二次多项式 $x^2 + x + \delta^{2^i}$, $i = 0, 1, \dots, m/2 - 1$, 得到余数矩阵 $\mathbf{U}(i)$, 由 $\mathbf{U}(i) = [\mathbf{U}(i)]^{2^i} = [\mathbf{U}]^{2^i}$, 可以得到

$$\mathbf{R} = \begin{bmatrix} \mathbf{U}(0) \\ \mathbf{U}(1) \\ \vdots \\ \mathbf{U}(m/2-1) \end{bmatrix} = \begin{bmatrix} \mathbf{U} \\ [\mathbf{U}]^2 \\ \vdots \\ [\mathbf{U}]^{2^{m/2-1}} \end{bmatrix}.$$

由公式(3.28), 有

$$\begin{aligned} \mathbf{R} &= \begin{bmatrix} \delta^0 & 0 & \delta^1 & \dots & \delta^{m/2-1} & 0 \\ 0 & \delta^0 & 0 & \dots & 0 & \delta^{m/2-1} \\ (\delta^{2^1})^0 & 0 & (\delta^{2^1})^1 & \dots & (\delta^{2^1})^{m/2-1} & 0 \\ 0 & (\delta^{2^1})^0 & 0 & \dots & 0 & (\delta^{2^1})^{m/2-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ (\delta^{2^{m/2-1}})^0 & 0 & (\delta^{2^{m/2-1}})^1 & \dots & (\delta^{2^{m/2-1}})^{m/2-1} & 0 \\ 0 & (\delta^{2^{m/2-1}})^0 & 0 & \dots & 0 & (\delta^{2^{m/2-1}})^{m/2-1} \end{bmatrix} \times \mathbf{B} \\ &= \mathbf{\Pi}^{-1} \times \left[\begin{array}{c|c} \mathbf{V}_{m/2} & \mathbf{O} \\ \hline \mathbf{O} & \mathbf{V}_{m/2} \end{array} \right] \times \mathbf{\Pi} \times \mathbf{B}. \end{aligned} \quad (3.30)$$

摩尔-范德蒙矩阵 $\mathbf{V}_k$ 可分解为以下形式,

$$\begin{aligned} \mathbf{V}_k &= \begin{bmatrix} (\alpha^{c_k 2^0})^0 & (\alpha^{c_k 2^0})^1 & \dots & (\alpha^{c_k 2^0})^{m-1} \\ (\alpha^{c_k 2^1})^0 & (\alpha^{c_k 2^1})^1 & \dots & (\alpha^{c_k 2^1})^{m-1} \\ \vdots & \vdots & \ddots & \vdots \\ (\alpha^{c_k 2^{m-1}})^0 & (\alpha^{c_k 2^{m-1}})^1 & \dots & (\alpha^{c_k 2^{m-1}})^{m-1} \end{bmatrix} \\ &= \left[\begin{array}{c|c} \mathbf{I}_{m/2} & \mathbf{O} \\ \hline \mathbf{I}_{m/2} & \mathbf{I}_{m/2} \end{array} \right] \times \left[\begin{array}{c|c} \beta^{2^0} & \\ & \beta^{2^1} \\ & & \beta^{2^2} \\ & & & \ddots \\ & & & & \beta^{2^{m/2-1}} \\ \hline \mathbf{O} & \mathbf{I}_{m/2} \end{array} \right] \times \\ &\quad \left[\begin{array}{c|c} \mathbf{V}_{m/2} & \mathbf{O} \\ \hline \mathbf{O} & \mathbf{V}_{m/2} \end{array} \right] \times \mathbf{\Pi} \times \mathbf{B}. \end{aligned} \quad (3.31)$$

综上, 定理 1 证明完毕。

当 $\mathbf{E}_{c_k}$ 为 m 阶单位矩阵时, $\mathbf{H}_{c_k}^T$ 为摩尔-范德蒙矩阵, 因此 Fedorenko^[68]提出的方法可看作是本文提出方法的一个特例。

当分解结果中 $\mathbf{V}_{m/2}$ 的阶数为偶数时, 可以递归地进行分解, 阶数为奇数时, 运用 Blahut 短循环卷积方法^[69]做进一步分解。

3.3 快速解码算法

本节设计一种用于有限域 $GF(2^m)$ 上的 (N, K) RS 码快速解码算法, m 为偶数。该算法基于定理 1 中对范德蒙矩阵的分解, 具体算法步骤如下:

步骤 1: 预处理。计算有限域 $GF(2^m)$ 上的分圆陪集 $C^{(k)}$, 根据分圆陪集构造 T 行 2^m 列的范德蒙矩阵 $\bar{\mathbf{H}}$,

$$\bar{\mathbf{H}} = [\mathbf{H}_0 \quad \mathbf{H}_{c_1} \quad \mathbf{H}_{c_2} \quad \dots \quad \mathbf{H}_{c_l}]. \quad (3.32)$$

接着, 由矩阵 $\bar{\mathbf{H}}$ 的前 N 列构成 (N, K) RS 码的校验矩阵。在接收码字向量 $\mathbf{r}$ 右侧添加长度为 $2^m - N$ 的零向量得到向量 $\mathbf{r}'$, 即

$$\mathbf{r}' = [\mathbf{r} \quad \mathbf{0}]. \quad (3.33)$$

根据矩阵 $\bar{\mathbf{H}}$, 将向量 $\mathbf{r}'$ 进行对应的划分, 得到

$$\mathbf{r}' = [\mathbf{r}_0 \quad \mathbf{r}_{c_1} \quad \mathbf{r}_{c_2} \quad \dots \quad \mathbf{r}_{c_l}]. \quad (3.34)$$

步骤 2: 根据定理 1 计算伴随式 $\mathbf{s}$, 具体计算公式如下,

$$\begin{aligned} \mathbf{s} &= \mathbf{r}_0 \times \mathbf{H}_0^T + \sum_{k=1}^l \mathbf{r}_{c_k} \times \mathbf{H}_{c_k}^T \\ &= \mathbf{s}_0 + \sum_{k=1}^l \mathbf{s}_{c_k}. \end{aligned} \quad (3.35)$$

步骤 3: 通过 LU (Lower-Upper) 分解算法^[70, 71]计算线性方程组 $\mathbf{e} \times \mathbf{H}_{dec}^T = \mathbf{s}$ 中的擦除码字向量 $\mathbf{e}$ 。

步骤 1 是预处理操作, 因此在编解码中不会带来额外的计算开销。在步骤 2 中, 通过基于范德蒙矩阵的分解计算伴随式 $\mathbf{s}$, 下面介绍具体的计算过程。

首先通过公式计算得到 $\mathbf{s}_0$,

$$\begin{aligned} \mathbf{s}_0 &= \mathbf{r}_0 \times \mathbf{H}_0^T \\ &= [r_0 \quad r_1] \times \begin{bmatrix} 1 & 0 & \dots & 0 \\ 1 & 1 & \dots & 1 \end{bmatrix} \\ &= [r_0 + r_1 \quad r_1 \quad \dots \quad r_1]. \end{aligned} \quad (3.36)$$

接着, 由定理 1, 求解 $\mathbf{s}_{c_k}$,

$$\begin{aligned}
 \mathbf{s}_{c_k} &= \mathbf{r}_{c_k} \times \mathbf{H}_{c_k}^T \\
 &= \mathbf{r}_{c_k} \times \left[\begin{array}{c|c} \mathbf{I}_{m_k/2} & \mathbf{O} \\ \hline \mathbf{I}_{m_k/2} & \mathbf{I}_{m_k/2} \end{array} \right] \times \\
 &\quad \left[\begin{array}{c|c} \beta^{2^0} & \\ \beta^{2^1} & \\ \beta^{2^2} & \\ \vdots & \\ \beta^{2^{m_k/2-1}} & \\ \hline \mathbf{O} & \mathbf{I}_{m_k/2} \end{array} \right] \times \\
 &\quad \left[\begin{array}{c|c} \mathbf{V}_{m_k/2} & \mathbf{O} \\ \hline \mathbf{O} & \mathbf{V}_{m_k/2} \end{array} \right] \times \mathbf{\Pi} \times \mathbf{B} \times \mathbf{E}_{c_k},
 \end{aligned} \tag{3.37}$$

其中, $k=1, 2, \dots, l$ 。根据公式(3.10)可知, 乘法运算次数与 m 相关。范德蒙矩阵 $\mathbf{H}$ 根据有限域 $GF(2^m)$ 上分圆陪集 $C^{(k)}$ 构造, 因此在给定的有限域构造 RS 码时, 计算伴随式 $\mathbf{s}$ 需要的乘法运算次数存在上限值。此外, 与二元矩阵 $\mathbf{B}$ 、 $\mathbf{E}_{c_k}$ 相乘仅需要进行加法运算操作。

步骤 3 中, 使用 LU 分解算法求解擦除码字所需的乘法运算次数和加法运算次数都为 $T(T-1)$ 。

现通过例子详细说明解码算法过程。在有限域 $GF(2^4)$ 上构造参数 $(N, K) = (9, 5)$ RS 码。设码字向量 $\mathbf{c} = [c_0 \ c_1 \ \dots \ c_8]$ 中 4 个符号 c_{e_i} , $i=0, 1, 2, 3$, 发生了擦除。 $\mathbf{r}$ 表示接收的码字向量, 令 $r_i = c_i$, $i \notin \{e_0, e_1, e_2, e_3\}$, 否则 $r_i = 0$ 。

在步骤 1 中, 计算有限域 $GF(2^4)$ 上所有分圆陪集 $C^{(k)}$, $k=0, 1, 2, 3, 4$,

$$\begin{aligned}
 C^{(0)} &= \{0\}, \\
 C^{(1)} &= \{1, 2, 4, 8\}, \\
 C^{(2)} &= \{3, 6, 12, 9\}, \\
 C^{(3)} &= \{7, 14, 13, 11\}, \\
 C^{(4)} &= \{5, 10\}.
 \end{aligned} \tag{3.38}$$

根据分圆陪集生成对应的 $\mathbf{H}_{c_k}$, 具体地,

$$\begin{aligned}
 \mathbf{H}_0 &= \begin{bmatrix} 1 & 1 \\ 0 & 1 \\ 0 & 1 \\ 0 & 1 \end{bmatrix}, \\
 \mathbf{H}_{c_1} &= \begin{bmatrix} 1 & 1 & 1 & 1 \\ \alpha & \alpha^2 & \alpha^4 & \alpha^8 \\ \alpha^2 & \alpha^4 & \alpha^8 & \alpha^{16} \\ \alpha^3 & \alpha^6 & \alpha^{12} & \alpha^9 \end{bmatrix}, \\
 \mathbf{H}_{c_2} &= \begin{bmatrix} 1 & 1 & 1 & 1 \\ \alpha^3 & \alpha^6 & \alpha^{12} & \alpha^9 \\ \alpha^6 & \alpha^{12} & \alpha^9 & \alpha^3 \\ \alpha^9 & \alpha^3 & \alpha^6 & \alpha^{12} \end{bmatrix}, \\
 \mathbf{H}_{c_3} &= \begin{bmatrix} 1 & 1 & 1 & 1 \\ \alpha^7 & \alpha^{14} & \alpha^{13} & \alpha^{11} \\ \alpha^{14} & \alpha^{13} & \alpha^{11} & \alpha^7 \\ \alpha^6 & \alpha^{12} & \alpha^9 & \alpha^3 \end{bmatrix}, \\
 \mathbf{H}_{c_4} &= \begin{bmatrix} 1 & 1 \\ \alpha^5 & \alpha^{10} \\ \alpha^{10} & \alpha^5 \\ 1 & 1 \end{bmatrix}. \tag{3.39}
 \end{aligned}$$

接着，通过矩阵 $\mathbf{H}_{c_k}$, $k = 0, 1, 2, 3, 4$ ，构造 4 行 16 列的范德蒙矩阵 $\mathbf{H}$ ，

$$\mathbf{H} = [\mathbf{H}_0 \ \mathbf{H}_{c_1} \ \mathbf{H}_{c_2} \ \mathbf{H}_{c_3} \ \mathbf{H}_{c_4}]. \tag{3.40}$$

$(N, K) = (9, 5)$ RS 码的校验矩阵由矩阵 $\mathbf{H}$ 的前 9 列构成。令 $\mathbf{r}' = [\mathbf{r} \ \mathbf{0}]$ ， $\mathbf{0}$ 是长度为 $2^4 - 9 = 7$ 的零向量，将 $\mathbf{r}'$ 进行划分，得到

$$\mathbf{r}' = [\mathbf{r}_0 \ \mathbf{r}_{c_1} \ \mathbf{r}_{c_2} \ \mathbf{r}_{c_3} \ \mathbf{r}_{c_4}], \tag{3.41}$$

$$\mathbf{r}_0 = [r_0 \ r_1],$$

$$\mathbf{r}_{c_1} = [r_2 \ r_3 \ r_4 \ r_5],$$

$$\mathbf{r}_{c_2} = [r_6 \ r_7 \ r_8 \ 0], \tag{3.42}$$

$$\mathbf{r}_{c_3} = \mathbf{r}_{c_4} = \mathbf{0}.$$

接着在步骤 2 中计算伴随式 $\mathbf{s}$ 。通过下列公式求得 $\mathbf{s}_0$ ，

$$\begin{aligned}
 \mathbf{s}_0 &= \mathbf{r}_0 \times \mathbf{H}_0^T \\
 &= [r_0 + r_1 \ r_1 \ r_1 \ r_1]. \tag{3.43}
 \end{aligned}$$

根据定理 1 对矩阵 $\mathbf{H}_{c_k}$, $k = 1, 2, 3, 4$ 进行分解。

由多项式

$$\theta(x) = \sum_{i=0}^3 \theta_i x^i \quad (3.44)$$

除以多项式 $x^2 + x + \delta$, $\delta = \alpha^{2^{4/2}+1} = \alpha^5 \in GF(2^{4/2})$ 得到余数矩阵,

$$\mathbf{U} = \begin{bmatrix} 1 & 0 & \alpha^5 & \alpha^5 \\ 0 & 1 & 1 & 1 + \alpha^5 \end{bmatrix}, \quad (3.45)$$

由公式(3.28), 有

$$\mathbf{U} = \begin{bmatrix} 1 & 0 & \alpha^5 & 0 \\ 0 & 1 & 0 & \alpha^5 \end{bmatrix} \times \mathbf{B}, \quad (3.46)$$

得到 4 阶可逆矩阵 $\mathbf{B}$ 构成如下:

$$\mathbf{B} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (3.47)$$

矩阵

$$\begin{aligned} \mathbf{H}_{c_1}^T &= \begin{bmatrix} \alpha^0 & \alpha^1 & \alpha^2 & \alpha^3 \\ \alpha^{0 \cdot 2} & \alpha^{1 \cdot 2} & \alpha^{2 \cdot 2} & \alpha^{3 \cdot 2} \\ \alpha^{0 \cdot 4} & \alpha^{1 \cdot 4} & \alpha^{2 \cdot 4} & \alpha^{3 \cdot 4} \\ \alpha^{0 \cdot 8} & \alpha^{1 \cdot 8} & \alpha^{2 \cdot 8} & \alpha^{3 \cdot 8} \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & \alpha & 0 \\ 0 & 1 & 0 & \alpha^2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \\ &\quad \begin{bmatrix} 1 & \alpha^5 & 0 & 0 \\ 1 & \alpha^{10} & 0 & 0 \\ 0 & 0 & 1 & \alpha^5 \\ 0 & 0 & 1 & \alpha^{10} \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \\ &\quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

对分解结果中的摩尔-范德蒙矩阵 $\mathbf{V}_2$ 进一步分解,

$$\mathbf{V}_2 = \begin{bmatrix} 1 & \alpha^5 \\ 1 & \alpha^{10} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & \alpha^5 \\ 0 & 1 \end{bmatrix}. \quad (3.48)$$

计算 $\mathbf{s}_{c_k}$, $k = 1, 2, 3, 4$,

$$\begin{aligned}
 \mathbf{s}_{c_1} &= \mathbf{r}_{c_1} \times \mathbf{H}_{c_1}^T \\
 &= [r_2 \ r_3 \ r_4 \ r_5] \times \begin{bmatrix} \alpha^0 & \alpha^1 & \alpha^2 & \alpha^3 \\ \alpha^{0 \cdot 2} & \alpha^{1 \cdot 2} & \alpha^{2 \cdot 2} & \alpha^{3 \cdot 2} \\ \alpha^{0 \cdot 4} & \alpha^{1 \cdot 4} & \alpha^{2 \cdot 4} & \alpha^{3 \cdot 4} \\ \alpha^{0 \cdot 8} & \alpha^{1 \cdot 8} & \alpha^{2 \cdot 8} & \alpha^{3 \cdot 8} \end{bmatrix} \\
 &= [r_2 \ r_3 \ r_4 \ r_5] \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix} \times \\
 &\quad \begin{bmatrix} 1 & 0 & \alpha & 0 \\ 0 & 1 & 0 & \alpha^2 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \times \\
 &\quad \begin{bmatrix} 1 & \alpha^5 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \alpha^5 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \\
 &\quad \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{3.49}
 \end{aligned}$$

类似地,

$$\mathbf{s}_{c_2} = \mathbf{r}_{c_2} \times \mathbf{H}_{c_2}, \tag{3.50}$$

其中, 二元矩阵 $\mathbf{E}_{c_2}$ 为

$$\mathbf{E}_{c_2} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 1 \end{bmatrix}. \tag{3.51}$$

由于 $\mathbf{r}_{c_3}$ 和 $\mathbf{r}_{c_4}$ 都是零向量, 因此有 $\mathbf{s}_{c_3} = \mathbf{s}_{c_4} = \mathbf{0}$ 。

通过公式 $\mathbf{s} = \mathbf{s}_0 + \mathbf{s}_{c_1} + \mathbf{s}_{c_2}$ 得到所求伴随式 $\mathbf{s} = [s_0 \ s_1 \ s_2 \ s_3]$ 。

在步骤 3 中, 使用擦除符号位置在校验矩阵中对应位置的向量组成 4 阶矩阵 $\mathbf{H}_{dec}$ 。通过 LU 分解算法求解线性方程组 $\mathbf{e} \times \mathbf{H}_{dec}^T = \mathbf{s}$ 中擦除向量 $\mathbf{e} = [c_{e_0} \ c_{e_1} \ c_{e_2} \ c_{e_3}]$ 。至此完成 (9, 5) RS 码码字中失效符号的修复。

解码过程中, 步骤 2 求伴随式 $\mathbf{s}$ 需要 $4 \times 2 = 8$ 次乘法运算和 $1 + 10 + 8 + 4 \times 2 = 27$ 次加法运算。此外, 步骤 3 求解线性方程组所需的乘法运算次数和加法运算次数都为

$4 \times (4 - 1) = 12$ 。因此，使用本文提出的解码算法修复有限域 $GF(2^4)$ 上 $(9, 5)$ RS 码码字擦除符号，其计算复杂度总共为 $8 + 12 = 20$ 次乘法运算和 $27 + 12 = 39$ 次加法运算。

算法 1 展示了基于范德蒙矩阵分解的 RS 码解码算法流程。

算法 1: 有限域 $GF(2^m)$ 上基于范德蒙矩阵分解的 RS 码解码算法 (m 为偶数)

Input: $\mathbf{c} = [c_0 \ c_1 \ \dots \ c_{N-1}]$, $\mathbf{r} = [r_0 \ r_1 \ \dots \ r_{N-1}]$, 擦除码字位置 $\{e_0, e_1, \dots, e_{T-1}\}$,

Output: $\mathbf{e} = [c_{e_0} \ c_{e_1} \ \dots \ c_{e_{T-1}}]$

- 1 $\beta \leftarrow \alpha^{c_k}$
- 2 $\mathbf{Sub}_0 = \left[\begin{array}{c|c} \mathbf{I}_{m/2} & \mathbf{O} \\ \hline \mathbf{I}_{m/2} & \mathbf{I}_{m/2} \end{array} \right]$, $\mathbf{Sub}_1 = \left[\begin{array}{c|c} \mathbf{I}_{m/2} & \text{diag}[\beta^{2^0} \ \beta^{2^1} \ \dots \ \beta^{2^{m/2-1}}] \\ \hline \mathbf{O} & \mathbf{I}_{m/2} \end{array} \right]$, $\mathbf{Sub}_3 = \left[\begin{array}{c|c} \mathbf{V}_{m/2} & \mathbf{O} \\ \hline \mathbf{O} & \mathbf{V}_{m/2} \end{array} \right]$,
- $\mathbf{\Pi}, \mathbf{B}, \mathbf{E}_{c_k}, k = 1, 2, \dots, l$ // 构造分解后的子矩阵
- 3 **for** $i = 0$ **to** $N - 1$ **do** // 接收向量 $\mathbf{r}$ 赋值
- 4 **while** $i \neq \{e_0, e_1, \dots, e_{T-1}\}$ **do**
- 5 $r_i \leftarrow c_i$
- 6 $r_i \leftarrow 0$
- 7 $s_0 \leftarrow r_0 + r_1$ // 计算 $\mathbf{s}$ 的第 0 个元素
- 8 **for** $i = 1$ **to** T **do** // 计算 $\mathbf{s}$ 第 1 至 $T - 1$ 个元素
- 9 $s_i \leftarrow r_1$
- 10 **for** $i = 1$ **to** l **do** // 计算 $\mathbf{s}_{c_i}$ 并将其与 $\mathbf{s}$ 相加以获得伴随式
- 11 $\mathbf{s}_{c_i} \leftarrow [r_{2+i \cdot c_i} \ r_{3+i \cdot c_i} \ \dots \ r_{m+i \cdot c_i}] \times \mathbf{Sub}_1 \times \mathbf{Sub}_2 \times \mathbf{Sub}_3 \times \mathbf{\Pi} \times \mathbf{B} \times \mathbf{E}_{c_i}$
- 12 $\mathbf{s} \leftarrow \mathbf{s} + \mathbf{s}_{c_i}$

算法 1: 有限域 $GF(2^m)$ 上基于范德蒙矩阵分解的 RS 码解码算法 (m 为偶数) (续)

13 solve $\mathbf{e} \times \mathbf{H}_{dec}^T = \mathbf{s}_0$ // 通过 LU 分解算法求解擦除向量 $\mathbf{e}$

14 return $\mathbf{e} = [c_{e_0} \ c_{e_1} \ \dots \ c_{e_{T-1}}]$ // 返回擦除码字向量

3.4 解码算法复杂度分析

本节对提出解码算法的复杂度进行分析, 并与 RM 变换算法^[31]的解码算法进行比较。

设有限域上分圆陪集 $C^{(k_1)}, C^{(k_2)}, \dots, C^{(k_s)}$ 中, 各个集合里包含的元素个数取值共有 s 种, 用 $m_{k_1}, m_{k_2}, \dots, m_{k_s}$ 表示, 且 $m_{k_1} = m$ 。令 t_{k_j} 表示分圆陪集中包含 m_{k_j} 个元素的分圆陪集个数, $1 \leq j \leq s$ 。 p_k 是一个正整数, 且有

$$p_k = 2^{m_{k_s}} - 1, \quad m_{k_s} < m. \quad (3.52)$$

设 $\phi(p_k)$ 为满足 $\gcd(p_k, x) = 1$ 的 x 个数, 并设 $\psi(p_k)$ 为满足以下公式的 x 个数,

$$\frac{p_k}{\gcd(p_k, x)} \neq 2^\epsilon - 1, \quad \epsilon | m, \quad (3.53)$$

其中 $x \in [1, p_k)$ 。则有

$$\begin{cases} t_{k_j} = \frac{\phi(p_k) + \psi(p_k)}{m_{k_j}}, \quad j = 2, 3, \dots, s, \\ t_{k_1} = \frac{2^m - \sum_{k=2}^s t_{k_j} \cdot m_{k_j} - 2}{m_{k_1}}. \end{cases} \quad (3.54)$$

$mul_{m_{k_s}}$ 和 $add_{m_{k_s}}$ 分别表示向量与 m_{k_s} 阶摩尔-范德蒙矩阵 $\mathbf{V}_k$ 做乘法运算时所需要的乘法次数和加法次数, m_{k_s} 为偶数。由定理 1, 有

$$\begin{cases} mul_{m_{k_s}} = 2mul_{m_{k_s}/2} + m_{k_s}/2, \\ add_{m_{k_s}} = 2add_{m_{k_s}/2} + h_a, \end{cases} \quad (3.55)$$

其中 h_a 是向量与 m_{k_s} 阶矩阵 $\mathbf{B}$ 相乘所需加法运算次数。表 3.1 列出了向量与 m_{k_s} 阶摩尔-范德蒙矩阵相乘的计算复杂度, $m_{k_s} = 1, 2, 3, 4, 5, 6, 8, 10, 12$ 。

显然, 与矩阵 $\mathbf{E}_{c_k}$ 相乘仅需要进行加法运算。设 mul_s 和 add_s 分别为计算参数 $(2^m, K)$ RS 码的伴随式 $\mathbf{s}$ 时的乘法运算和加法运算次数, 由定理 1 和公式(3.37)可以推

出：

表 3.1 与有限域 $GF(2^{m_{k_s}})$ 上摩尔-范德蒙矩阵相乘的计算复杂度

m_{k_s}	h_a	add_{m_k}	mul_{m_k}
1	0	0	0
2	0	2	1
3	-	9	3
4	3	11	4
5	-	40	9
6	8	29	9
8	16	42	12
10	23	94	23
12	36	100	24

$$\begin{cases} mul_s = \sum_{j=1}^s t_{k_j} mul_{m_{k_s}}, \\ add_s = \sum_{j=1}^s (t_{k_j} add_{m_{k_s}} + h_{m_k}) + T \sum_{j=1}^s t_{k_j} + 1, \end{cases} \quad (3.56)$$

其中 h_{m_k} 是与每个矩阵 $\mathbf{E}_{c_k}$ 相乘所需加法运算次数的和。

此外，LU 分解算法^[70, 71]求解线性方程组计算复杂度为

$$\begin{cases} mul_{LU} = T(T-1), \\ add_{LU} = T(T-1). \end{cases} \quad (3.57)$$

由以上公式，本文提出的解码算法计算复杂度为

$$\begin{cases} mul_{total} = mul_s + mul_{LU}, \\ add_{total} = add_s + add_{LU}. \end{cases} \quad (3.58)$$

RM 变换算法^[31]在解码过程中乘法运算次数为

$$1 + \sum_{i=1}^{T-1} \sum_{j=1}^{b(i)} \left(\frac{\log_2^N}{j} \right) + T^2 - 2T, \quad (3.59)$$

其中 T 为 RS 码的冗余， $b(i)$ 是 i 的二进制表示中“1”的个数。表 3.2-3.5 分别列出了本文提出的编解码算法和 RM 变换算法^[31]在有限域 $GF(2^4)$ 上进行解码所需的乘法运算

次数和加法运算次数的对比。可以看出, 当 $T = 4, 5, 6, 7$, 相比 RM 变换算法, 本文提出的算法在解码时需要的乘法运算次数更少。若码长 $N = 16$, 当 $T = 4$ 和 $T = 7$ 时, 本文提出的算法在解码时所需乘法次数相比 RM 变换算法分别减少了 7% 和 29%。这是因为本文的算法在计算伴随式 $\mathbf{s}$ 时, 所需乘法运算次数存在一个上限值, RM 变换算法需要的乘法次数随着冗余 T 的增加而增加。

表 3.2 有限域 $GF(2^4)$ 上不同 RS 码擦除解码算法加法复杂度对比 ($T=4, 5$)

N	$T=4$			$T=5$		
	RM 变换算法 ^[31]	本文算法	降低幅度	RM 变换算法 ^[31]	本文算法	降低幅度
8	35	37	-12%	-	-	-
9	55	39	29%	66	52	21%
10	56	41	27%	67	54	19%
11	57	48	16%	68	62	9%
12	57	52	9%	68	66	3%
13	58	54	7%	69	68	1%
14	58	56	3%	69	70	-1%
15	58	61	-5%	69	76	-10%
16	58	63	-9%	69	78	-13%

表 3.3 有限域 $GF(2^4)$ 上不同 RS 码擦除解码算法加法复杂度对比 ($T=6, 7$)

N	$T=6$			$T=7$		
	RM 变换算法 ^[31]	本文算法	降低幅度	RM 变换算法 ^[31]	本文算法	降低幅度
10	84	66	21%	-	-	-
11	86	75	13%	106	91	14%
12	86	79	8%	106	95	10%
13	88	81	8%	109	97	11%
14	88	83	6%	109	99	9%
15	88	90	-2%	109	107	2%
16	88	92	-5%	109	109	0%

表 3.4 有限域 $GF(2^4)$ 上不同 RS 码擦除解码算法乘法复杂度对比 ($T=4, 5$)

N	$T=4$			$T=5$		
	RM 变换算法 ^[31]	本文算法	降低幅度	RM 变换算法 ^[31]	本文算法	降低幅度
8	21	19	10%	-	-	-
9	24	20	17%	35	28	20%
10	25	20	20%	36	28	22%
11	26	23	12%	37	31	16%
12	26	23	12%	37	31	16%
13	27	24	11%	38	32	16%
14	27	24	11%	38	32	16%
15	27	25	7%	38	33	13%
16	27	25	7%	38	33	13%

表 3.5 有限域 $GF(2^4)$ 上不同 RS 码擦除解码算法乘法复杂度对比 ($T=6, 7$)

N	$T=6$			$T=7$		
	RM 变换算法 ^[31]	本文算法	降低幅度	RM 变换算法 ^[31]	本文算法	降低幅度
10	53	38	28%	-	-	-
11	55	41	25%	75	53	29%
12	55	41	25%	75	53	29%
13	57	42	26%	78	54	31%
14	57	42	26%	78	54	31%
15	57	43	25%	78	55	29%
16	57	43	25%	78	55	29%

3.5 本章小结

在本章中，首先对多点求值和归一化循环卷积相关概念作出了介绍。接着根据分圆陪集构造了一种新的范德蒙矩阵，并以此设计了一种在有限域上构造的 RS 码。然后设计了范德蒙矩阵分解算法，将范德蒙校验矩阵分解成多个矩阵相乘的方式并给出详

细证明了分解方法的正确性。基于此，本章提出了一种 RS 码的快速解码算法，有效减少了在 RS 码解码过程的乘法运算次数，降低了计算复杂度。最后，与基于 RM 变换的解码算法进行了在具体有限域 $GF(2^4)$ 上构造的 RS 码的计算复杂度对比分析，从结果看出，码长 $N=16$ ，当 $T=4$ 和 $T=7$ 时，本文提出的算法在解码时所需乘法次数比 RM 变换算法分别减少了7%和29%，具有更低的计算复杂度。

第四章 系统的设计与实现

在本章中，我们从系统结构设计、内部各节点（主节点以及从节点等）功能、工作流程原理等方面介绍了本文所搭建的改进 Hadoop 分布式文件系统（HDFS）。改进 HDFS 在保留了原本功能的同时，增加了能够搭载编解码算法的代理节点。基于此，本章后面部分对前文提出的 RS 码编解码算法进行了性能测试，并与其他算法比较分析。

4.1 系统简介

受谷歌文件系统启发而设计开发出的 Hadoop 分布式文件系统（HDFS）是 Apache Hadoop 生态中的关键组件，其为在分布式存储系统中实现集群化部署以应对大数据高效存储问题提供开放源代码技术方案。同时，HDFS 具有支持定制化的特性，这为在系统中引入代理节点（ProxyNode）实现编解码组件模块以及文件版本控制模块深度集成提供了可能性。这种新型的系统架构规避了核心组件重构风险，保障原系统的拓扑稳定性，另一方面新增的功能模块以独立的形态部署，系统部署维护变得简单易行。在网络编码云存储系统（Network Coding-based Cloud Storage System, NCCloud）^[72]中，架构模型的核心思想是基于代理。结合 HDFS 以及 NCCloud 两者的特点，本文在设计改进 HDFS 时设置了代理节点。

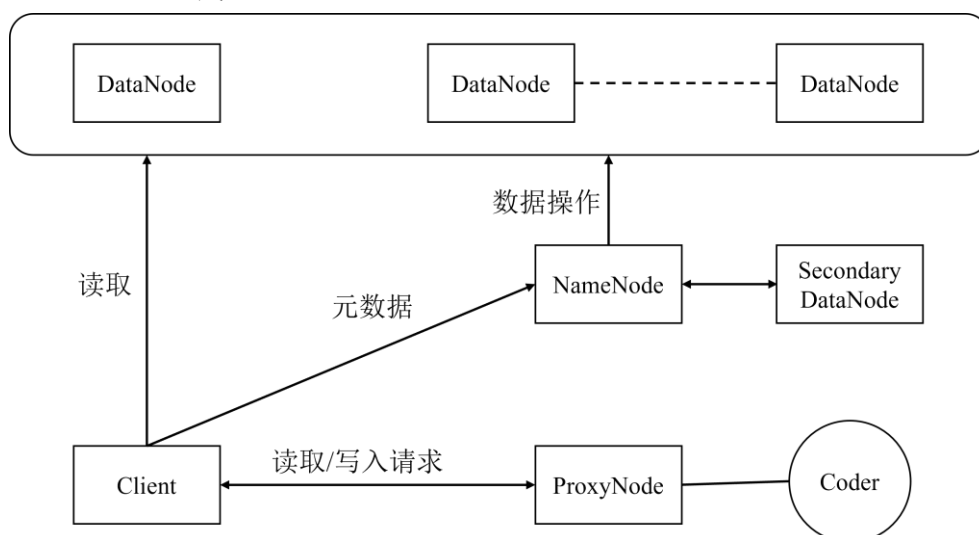


图 4.1 改进 HDFS 系统核心组件及原理示意图

图 4.1 中展示了改进 HDFS 系统核心组件以及原理。系统的数据处理流程中，核心机制如下：内置在代理节点中的算法组件对原始文件执行纠删码运算，生成带有冗余

校验码的编码数据包，这些封装后的编码数据经过安全链路返回给客户端，客户端基于标准化 API 接口向集群发起数据传输请求。系统采用计算-存储分离架构，从物理上将编解码组件和分布式存储组件隔离开，具体表现在减少跨组件依赖风险的功能解耦以及降低改进异构模块的开发成本。在改进 HDFS 中，各节点功能作用如下：

(1) 主节点 (NameNode)：单个实例管理中枢，承担协调集群、管控资源双重职能，具体的核心职责包括：命名空间治理，即通过层级化目录结构对元数据访问实施控制、维护元数据路径与物理存储的映射关系；存储策略配置，在本文中设置为单副本模式；数据块元数据管理，记录文件数据分块在数据节点上的物理分布信息；多角色通信枢纽，响应客户端文件传输请求时建立控制通道，在编解码阶段与代理节点协同。

(2) 辅助节点 (SecondaryNameNode)：定位为管理中枢的辅助，通过定期触发检查点操作，将主节点产生的 fsimage 快照文件以及 edit 日志文件进行合并后推送到主节点。

(3) 从节点 (DataNode)：物理存储执行单元，具体的核心职责包括：依据控制指令（来自客户端或主节点），执行存储单元的创建、更新及清除操作，对数据块的物理层生命周期实施管控；通过心跳协议周期性向主节点发送节点存活状态、块完整性以及容量等关键指标；为主节点提供块定位服务，为客户端提供数据直接存取接口。

(4) 客户端 (Client)：提供用户与分布式存储系统数据交互的 API。

(5) 代理节点 (ProxyNode)：本文在 HDFS 分布式文件系统中额外添加的用于实现编解码操作的节点，搭载本文提出的基于范德蒙矩阵分解的 RS 码快速解码算法。

在 HDFS 实现技术栈中，Java 作为核心开发语言，构建了标准化的、涵盖文件传输控制、元数据操作等预置模块的功能组件库，另外开放 C 语言接口以支持异构编程。

本文研究 RS 码的快速解码算法，在算法运行时涉及高密度异或操作以及乘法操作，因此本文引入了 Intel 公司的开源加速库 ISA-L，从底层指令集层面优化运算速度。本文将 RS 码的解码算法封装到系统 Coder 组件中。

4.2 系统工作流程

本小节阐述改进 HDFS 分布式文件系统的核心操作流程，涵盖系统中数据文件上传、下载及编解码操作。

4.2.1 上传文件

在 HDFS 中引入代理节点，实现了编码计算和数据存储的解耦，文件上传流程发生了改变：原始文件经由代理节点中的 RS 码算法组件执行编码运算，生成带有校验信

息的数据块，之后数据块通过安全链路回传到客户端，客户端调用预设的接口传输经编码处理后的数据到存储集群中存放。图 4.2 展示了文件上传的流程，整体分成以下 8 个步骤：

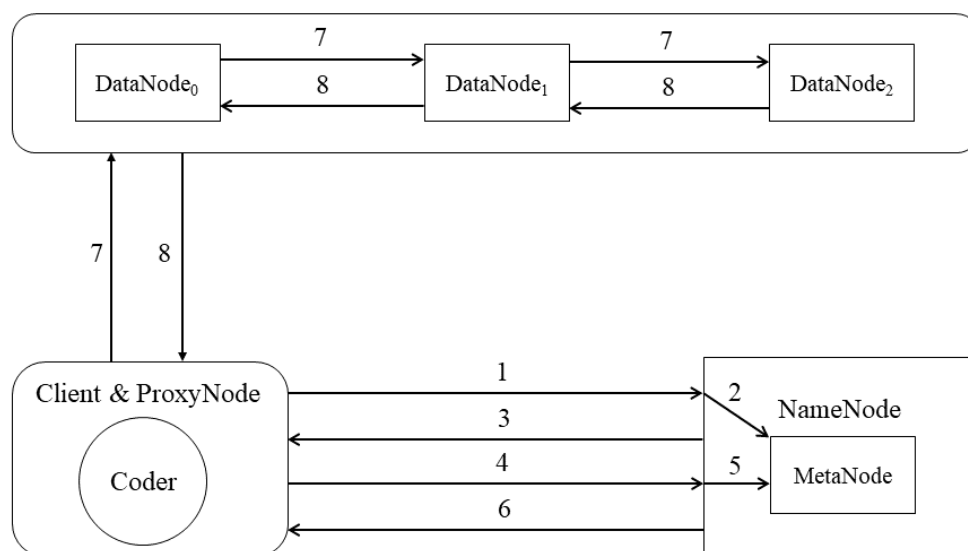


图 4.2 改进 HDFS 文件上传原理示意图

(1) 数据写入请求发起：客户端通过通信协议向主节点发起请求，系统基于远程过程调用（Remote Procedure Call, RPC）机制构建双向安全通信链路。

(2) 元数据冲突检测与权限验证：主节点接收到写入请求后，执行元数据命名唯一性校验和分布式权限验证框架双重验证机制，前者对在元数据目录树中执行确保未与现有文件重名，后者基于核查客户端写入权限。

(3) 元数据条目初始化：目录树中无同名元数据记录，则执行创建新元数据条目以及赋予全局唯一元数据标识符操作，之后经 RPC 响应通知客户端启动数据传输。

(4) 存储策略协调同步：客户端向主节点同步数据块参数，包括冗余副本数量（本文 RS 纠删码为单副本模式）以及数据块单个尺寸（确保与存储系统要求一致）。

(5) 存储资源调度分配：主节点基于负载均衡策略以及拓扑感知策略为元数据分配可用从节点。

(6) 从节点地址信息返回：主节点将候选从节点的网络标识（IP 地址）告知客户端。

(7) 元数据传输通道建立：根据接收的从节点地址列表，客户端与从节点之间建立点对点单向传输链路，基于标准协议推送元数据包（Packet）。

(8) 元数据分块写入与确认：客户端按照既定的数据包尺寸（例如 64KB/包）将元数据分阶段上传至从节点：首个元数据编码块被推送到首跳从节点存储；按条件进

行副本传输，具体地，在从节点收到数据后，链式转发到下一个节点（直到满足所设副本数），每完成一次节点数据写入，都需逆向发送确认信息（Acknowledgement, ACK）到上游节点，ACK 超时未送达则触发数据重传事件。完成当前块的传输后，客户端请求主节点分配下一个从节点。重复执行步骤 4 到 8 直到全部编码块写入到集群中。

4.2.2 下载文件

图 4.3 展示了改进 HDFS 下载文件的原理，客户端可以通过特定 API 接口的调用，实现从改进 HDFS 中下载文件的需求，可将其下载文件的流程大致分为 5 个步骤：

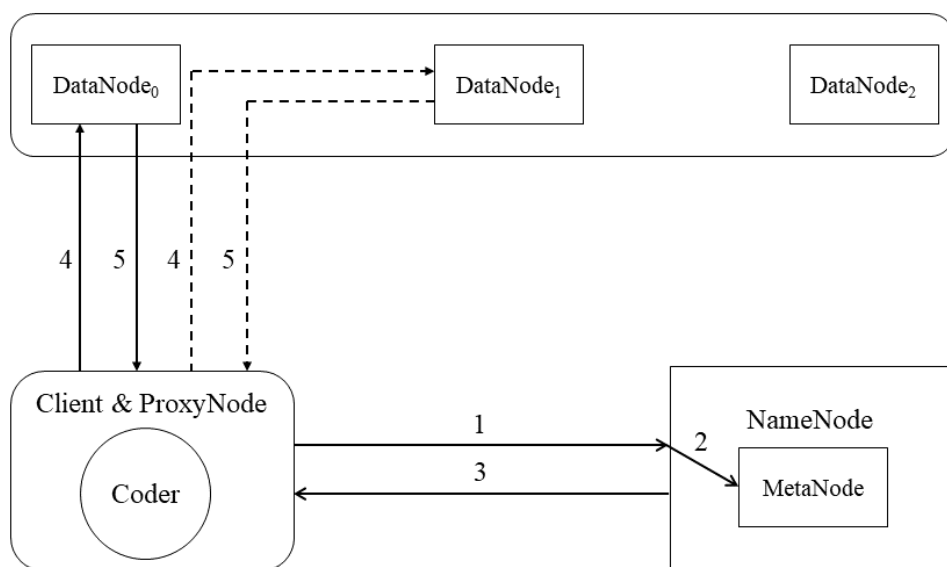


图 4.3 改进 HDFS 下载文件原理示意图

- （1）文件下载会话初始化：客户端通过 RPC 向主节点发起文件数据读取请求，触发加密通信链路动作。
- （2）元数据检索与拓扑映射：主节点在目录树中遍历定位到目标文件数据到块映射表、根据块标识符检索对应从节点物理位置，之后生成从节点列表。
- （3）存储路径反馈：主节点将元数据块位置列表、副本优先级排序等信息返回客户端。
- （4）数据传输链路构建：客户端根据基于网络拓扑感知的路由权重等参数首选同机架或者低延迟的从节点，激活流式传输协议，完成传输链路构建。
- （5）数据正确性校验及完整性保障：客户端以包为单位接收元数据，并实时对数据进行验证。在完整性验证过程中，比对接收端计算出的校验值与原始值，如果出现不一致，则将当前数据块标记为失效并触发备用数据切换机制，向主节点发送请求获取下一组块。客户端按顺序将数据块进行重组得到完整文件。

以上下载文件流程通过动态路由选择和分层校验同时实现数据完整性保障与跨节

点数据高效传输。

4.2.3 执行文件编解码算法

在改进 HDFS 中，核心在于将编解码算法模块与存储模块相互隔离，通过新增的中继节点对存储系统的功能实现了改进。系统架构方面，采用分层处理机制进行编码操作：文件通过分割预处理后进行条带化映射，随后执行 RS 纠删码编码运算生成冗余校验，最终调用存取接口实现文件元数据在系统集群中的存储与读取。在具体的实现中，将传输的一组基本数据块包的长度设置为固定的值。

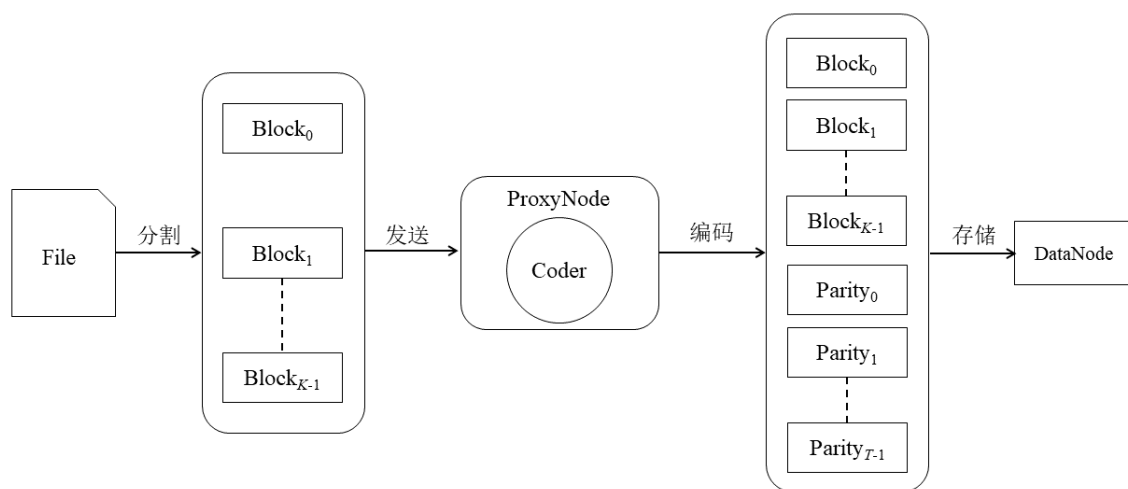


图 4.4 改进 HDFS 编码过程原理示意图

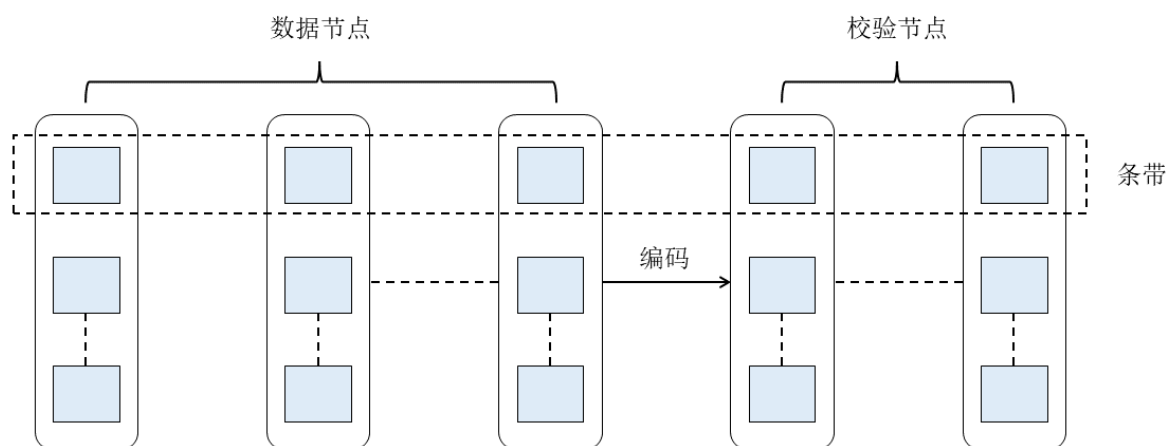


图 4.5 改进 HDFS 文件划分条带化原理示意图

在图 4.4 展示了文件编码过程，并在图 4.5 给出文件分割条带化示意。原始文件按照预设的算法被分割成 K 个等长的数据块后，在 K 个从节点中保存，记为

$\text{Block}_0, \text{Block}_1, \dots, \text{Block}_{K-1}$ 。按照编码算法得到 T 个校验块,并存储在 T 个校验节点中,记为 $\text{Parity}_0, \text{Parity}_1, \dots, \text{Parity}_{T-1}$ 。编解码和更新过程的基本单元是条带,由一次编码过程中出现的数据块和校验块合并组成,条带与条带之间的操作是相互独立的。

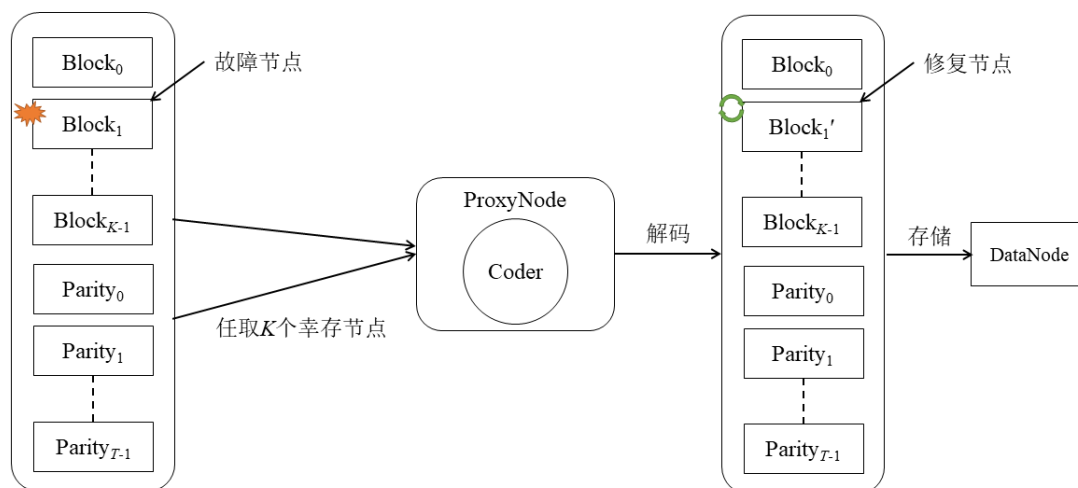


图 4.6 改进 HDFS 系统解码过程原理示意图

如图 4.6 所示,在解码过程中,从 N 个节点中选取任意的 K 个幸存节点,将对应的数据同步到系统代理节点 ProxyNode,并在节点搭载的解码模块中进行运算,重建由于节点宕机或网络隔离而不可用的数据,之后再将正确的数据上传到新节点中存储。

4.3 实验测试

在第三章中详细介绍了基于范德蒙矩阵分解的 RS 码快速解码算法,并对其计算复杂度进行了推导。为验证本文提出 RS 码算法在工程实践中的效能,本研究基于阿里云服务器部署了轻量化改进 HDFS 原型,并将本文提出的基于范德蒙矩阵分解的 RS 码算法模块搭载到代理节点 ProxyNode 中。实验测试了不同参数条件下本文提出算法的编解码性能,并将其和基于 RM 变换的算法、存储系统中常用的可运行在支持单指令多数据流(Single Instruction Multiple Data, SIMD)技术处理器上的 Jerasure2.0^[73]、ISA-L^[74]开源库进行了对比。

4.3.1 实验内容说明

对 RS 码性能进行评估时,重点关注其编码计算复杂度与解码计算复杂度等核心指标,对这些核心指标的量化分析通常情况是从时间角度出发。具体而言,编码处理的时长反映出由原始数据计算获得冗余校验数据块的复杂度,更短的耗时意味着更低的复杂度。而解码重构数据的耗时表征从 K 个幸存数据块恢复 N 个原始数据块的运算效率。需特别说明的是,实验中所有的 RS 码构造在以 $F(x) = x^8 + x^4 + x^3 + x^2 + 1$ 作为

本原多项式的有限域 $GF(2^8)$ 上,实验结果取算法运行 12 次后的平均值,其中解码时长指标是对所有可能节点失效场景的重构耗时之和。表 4.1 列出了性能的具体量化指标,表 4.2 列出了实验相关的参数变量。

表 4.1 实验性能指标

指标名称	定义	单位
编码耗时	数据块经过编码计算生成校验块消耗的时间	s
编码速度	原始文件体积/编码耗时	MB/s
解码耗时	幸存编码块经过解码计算恢复失效数据块消耗的时间	s
解码速度	原始文件体积/解码耗时	MB/s

表 4.2 实验参数变量

参数	说明
N	单个码字长度,即信息块和校验块之和
K	信息块个数
T	校验块个数
PacketSize	单个数据块体积,即最小数据单位
DataSize	原始文件体积

4.3.2 实验环境配置

本次实验中的 PC 硬件配置如下:处理器采用英特尔 Core i3-8100 3.6GHz,内存模组为芝奇 8GB×2 DDR4 2400MHz 双通道内存,以及拥有 1TB 容量的致钛 SSD 固态硬盘。软件层面,本次实验在 Ubuntu 22.04 操作系统构建,开发环境中配置 GCC 8.3.0 与 ISA-L 加速库,使用 C 语言作为软件开发语言,对比测量不同参数下基于 RM 变换的算法、Jerasure2.0 和 Intel 的 ISA-L 的编解码速度。

本文搭建了包含多个节点的改进 HDFS 作为分布式集群仿真环境,其中各个节点信息如下:

主节点:部署在 183.15.4.102 中的主节点服务器,承担集群协调、管控资源职能,由层级化目录树结构的方式实施对元数据管理,作为集群多个角色的通信枢纽协调各节点工作。

辅助节点:位于 183.15.4.122,作为主节点(管理中枢)的辅助,承担在元数据恢

复过程提供支持的职责。

从节点：从节点（183.15.4.125-127）作为物理存储单元，通过标准化协议周期性上传数据块状态报告。

客户端：部署在 183.15.4.133 的客户端作为用户操作接口为数据的存取提供支持。表 4.3 列出了各个节点的配置信息。

表 4.3 改进 HDFS 节点信息

地址	角色	名称
183.15.4.102	NameNode	Admin
183.15.4.122	SecondNameNode/ProxyNode	Proxy
183.15.4.125	DataNode ₀	Sla ₀
183.15.4.126	DataNode ₁	Sla ₁
183.15.4.127	DataNode ₂	Sla ₂
183.15.4.133	Client	Desktop

4.4 实验结果分析

4.4.1 编码性能

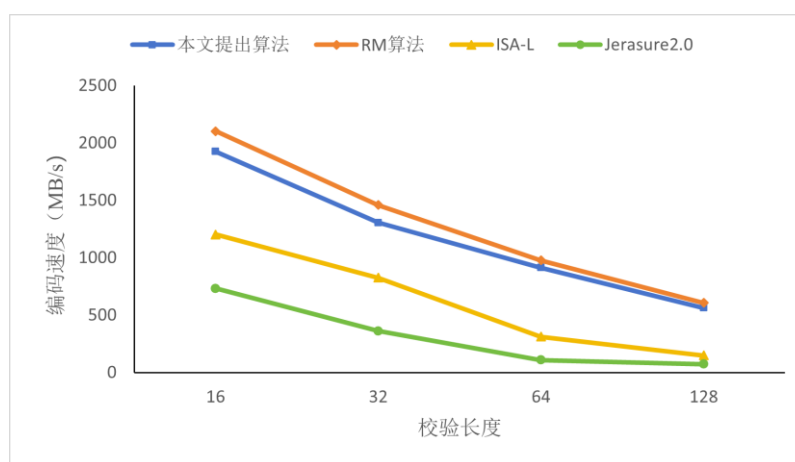


图 4.7 $T = 16, 32, 64, 128$ 时编码速度变化

(1) 测试不同校验长度 T 的条件下各算法的编码性能。原始文件大小 $FileSize=256MB$ ，单个数据块大小 $PacketSize=2048Byte$ ，RS 码码字长度 $N=256$ 。各算法编码速度见图 4.7。从结果看出，随着校验长度 T 取值的增加，各算法编码速度均有所下降。当 $T = 16$ 时，本文提出的算法编码速度为 1926 MB/s，当 $T = 128$ 时，编码速

度为 562 MB/s。

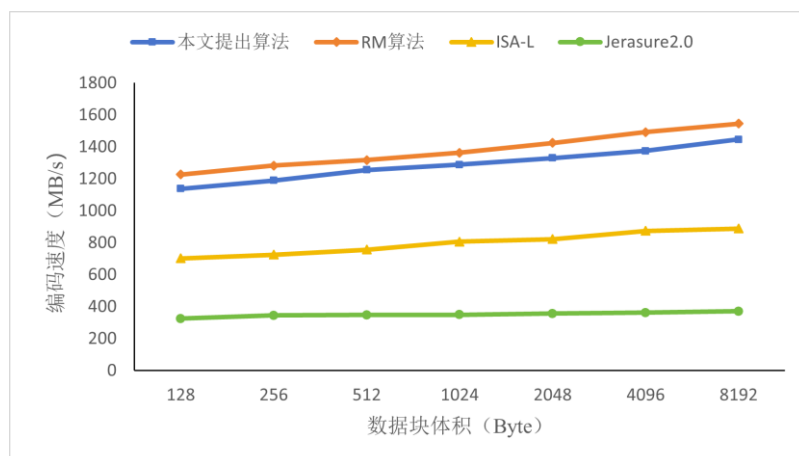


图 4.8 PacketSize = 128, 256, 512, 1024, 2048, 4096, 8192Byte 时编码速度变化

(2) 测试不同数据块大小 PacketSize 对编码性能的影响。实验测试原始文件大小 DataSize=256MB，码长 $N=256$ ，校验长度 $T=32$ ，数据块大小 PacketSize=128, 256, 512, 1024, 2048, 4096, 8192Byte。各算法编码性能如图 4.8 所示，随着数据块的增大，算法的性能均得到了提升。与 PacketSize=128Byte 相比，当 PacketSize=8192Byte 时，本文提出算法编码速度由 1138MB/s 提升至 1446MB/s，提升了 21%。在实际生产中，可通过增加数据块的大小提升系统的编码性能。

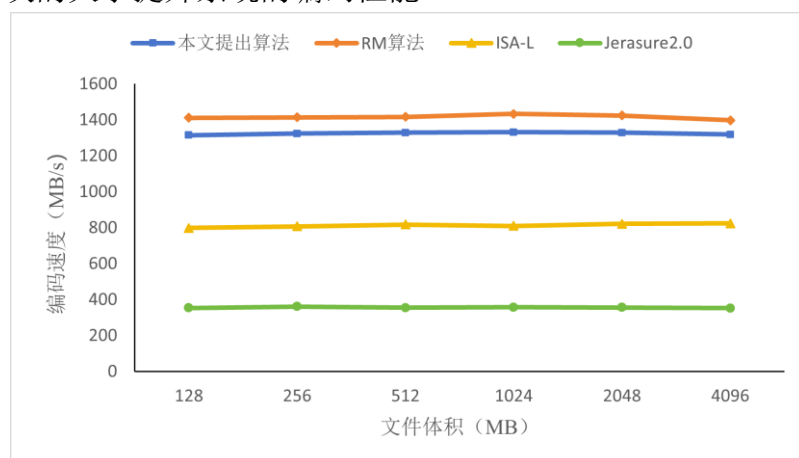
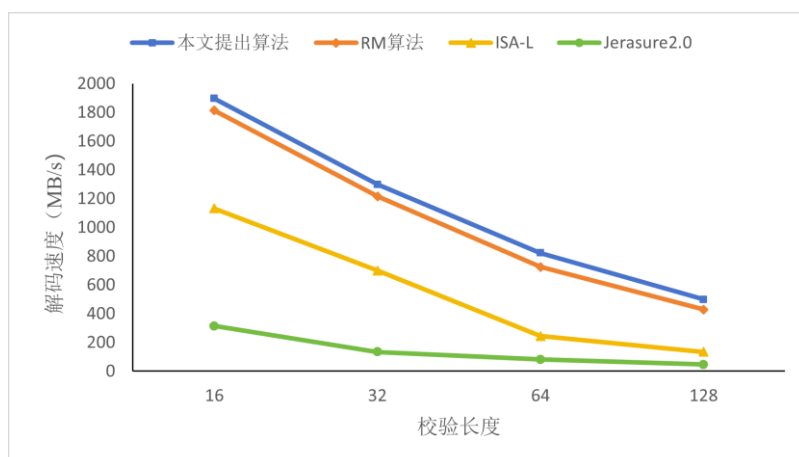


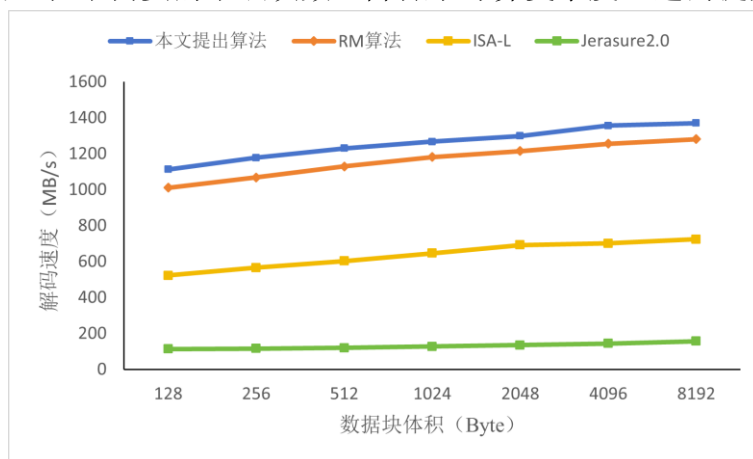
图 4.9 DataSize = 128, 256, 512, 1024, 2048, 4096MB 时编码速度变化

(3) 测试不同文件体积 DataSize 对编码性能的影响。实验测试单个数据块大小 PacketSize=2048Byte，码长 $N=256$ ，校验长度 $T=32$ 。从图 4.9 看出，当文件体积 DataSize 由 128MB 逐渐增加到 4096MB 时，4 种 RS 码算法的编码速度未发生明显变化，本文提出算法编码速度范围为 1315MB/s 至 1332MB/s。这表明文件体积大小对 RS 码编码速度未产生显著影响。

4.4.2 解码性能

图 4.10 $T = 16, 32, 64, 128$ 时解码速度变化

(1) 测试不同校验长度 T 的条件下各算法的解码性能。原始文件大小 $\text{DataSize}=256\text{MB}$, 单个数据块大小 $\text{PacketSize}=2048\text{Byte}$, RS 码码字长度 $N=256$ 。从图 4.10 的结果看出, 当 $T=16, 32, 64, 128$ 时, 本文提出的算法具有最优的解码性能, 相比 RM 算法分别提升了 5%, 7%, 13%, 17%, 相比 ISA-L 分别提升了 68%, 86%, 237%, 271%, 相比 Jersure2.0 分别提升了 504%, 864%, 896%, 976%。本文提出的算法减少了解码过程中需要的乘法次数, 降低了计算复杂度, 进而提升了解码性能。

图 4.11 $\text{PacketSize} = 128, 256, 512, 1024, 2048, 4096, 8192\text{Byte}$ 时解码速度变化

(2) 测试不同数据块大小 PacketSize 对解码性能的影响。实验测试原始文件大小 $\text{DataSize}=256\text{MB}$, 码长 $N=256$, 校验长度 $T=32$ 。从图 4.11 的结果可以看出, 同编码性能变化趋势相似, 各算法解码性能随着 PacketSize 的增加而提升。与 $\text{PacketSize}=128\text{Byte}$ 相比, 当 $\text{PacketSize}=8192\text{Byte}$ 时, 本文提出的算法解码速度由 1113MB/s 提升至 1371MB/s , 提升了 23%。这表明设定一个较大的 PacketSize 对分布式存储系统总体解

码性能是有帮助的。

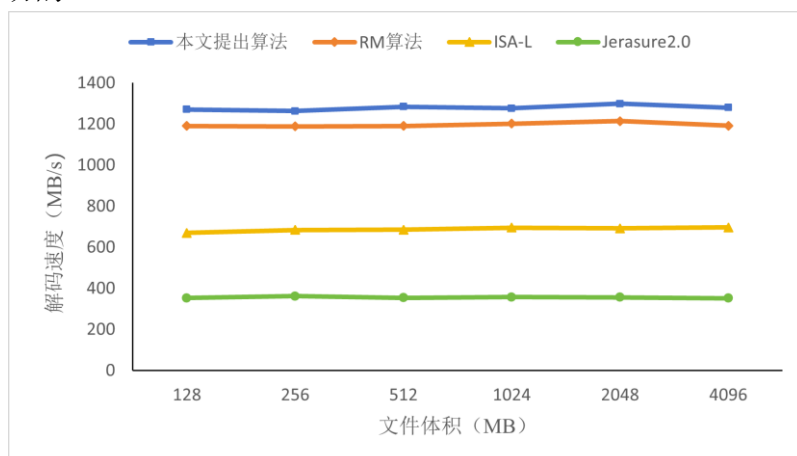


图 4.12 DataSize = 128, 256, 512, 1024, 2048, 4096MB 时解码速度变化

(3) 测试不同文件体积 DataSize 对解码性能的影响。实验测试单个数据块大小 PacketSize=2048Byte, 码长 $N=256$, 校验长度 $T=32$ 。从图 4.12 看出, 当文件体积 DataSize 由 128MB 逐渐增加到 4096MB 时, 4 种算法的解码速度变化不明显, 其中本文提出的算法解码速度范围为 1263MB/s 至 1299MB/s, 这表明本文提出的算法适用于大体积文件存储。

4.5 本章小结

本章首先搭建了一个改进的 HDFS 用于实验测试本文提出算法的性能, 该系统在常规的 HDFS 上增加一个挂载 RS 码算法的代理节点。在对系统原理进行介绍后完成了本文提出的算法与 RM 算法、ISA-L 和 Jersure2.0 在不同参数下的编解码性能对比。结果表明, 得益于解码过程中所需乘法次数的减少, 当 $N=256$, 相比 RM 算法、ISA-L 和 Jersure2.0, 本文提出的算法在 $T=16, 32, 64, 128$ 时具有最优的解码速度, 且解码速度优势随着 T 的增加而增加, 其分别提升了 5%~17%、68%~271% 和 504%~976%。增加数据块大小有助于提升系统编解码速度, 与 PacketSize=128Byte 相比, 当 PacketSize=8192Byte 时, 本文提出的算法编码速度由 1138MB/s 提升至 1446MB/s, 提升了 21%, 解码速度由 1113MB/s 提升至 1371MB/s, 提升了 23%。

第五章 总结与展望

5.1 本文总结

信息技术的飞快发展,在提升人们生活水平的同时,每时每刻都在产生庞大的数据量。使用纠删码作为容错策略的分布式存储系统是解决这些海量数据存储问题的最佳选择,原因在于系统兼具低存储成本与高可靠性特点。RS 码是一类构造在有限域上并得到广泛应用的 MDS 纠删码,但其在解码过程中需要进行大量带来高计算复杂度的有限域乘法操作,由此导致了性能瓶颈。针对该问题,本文对 RS 码纠删码技术进行研究,设计了一种新的 RS 码结构,并提出了一种基于范德蒙矩阵分解的快速解码算法以此提升 RS 码解码性能,主要的工作内容如下:

(1) 提出了一种新的构造在有限域 $GF(2^m)$ (m 为偶数)上的 RS 码结构,该 RS 码使用基于分圆陪集构造的范德蒙矩阵作为校验矩阵。之后设计了一种范德蒙矩阵分解方法,并基于此方法提出了能够减少 RS 码解码过程中所需乘法次数进而提升解码性能的快速解码算法。随后对提出算法的计算复杂度进行了推导,并与 RM 变换算法进行对比,在有限域 $GF(2^4)$ 上,当 $N=16$, $T=4$ 和 $T=7$ 时,本文提出算法解码所需乘法次数分别减少 7%和 29%。

(2) 参考云存储中的中继思想,本文根据纠删码相关理论,通过添加 1 个代理节点 ProxyNode 对常规 HDFS 进行改进,之后将本文提出的 RS 码算法搭载到系统中。在改进的 HDFS 上测试了本文提出算法在不同参数条件下的性能表现。从实验结果看出,在有限域 $GF(2^8)$ 上,当 $N=256$, $T=16, 32, 64, 128$ 时,本文提出的算法在具有最优的解码速度,相比 RM 算法、ISA-L 和 Jerasure2.0,分别提升了 5%~17%、68%~271% 和 504%~976%。此外,与数据块大小 PacketSize=128Byte 相比,当 PacketSize=8192Byte 时,本文提出的算法编码速度和解码速度分别得到 21%和 23%的提升。

5.2 未来展望

本文提出的 RS 码快速解码算法构造有限域 $GF(2^m)$ (m 为偶数)上,其具有解码过程乘法运算次数少、计算复杂度较低的特点,解码性能优于现有其他 RS 码算法,但还存在一些可以改进的空间:

(1) 解码算法仅适用与二元有限域的偶数幂次扩域上,面对日益变高的数据存储要求,构造 RS 码的有限域选择需要更加灵活。未来可以研究在奇数幂次扩域上类似的解码算法。

(2) 本文构造的是有限域上的 RS 码，在计算过程中仍然需要乘法运算，这无疑仍然一定程度限制了 RS 码性能。而在循环环上构造 RS 码，只需要进行异或及循环移位运算，未来可以探索将算法在环上运用的可能性。

(3) 实际分布式存储系统很多都是由多个机架构成，当前研究仅聚焦在单一机架，未涉及机架间的数据传输（修复带宽），研究降低 RS 码跨机架修复带宽拥有较高实际价值。

参考文献

- [1] Tomashevskaya V S, Yakovlev D A. Research of unstructured data interpretation problems[J]. Russian Technological Journal, 2021, 9(1): 7-17.
- [2] Javadpour A, Abadi A M H, Rezaei S, et al. Improving load balancing for data-duplication in big data cloud computing networks[J]. Cluster Computing, 2022, 25(4): 2613-2631.
- [3] Naeem M, Jamal T, Diaz-Martinez J, et al. Trends and future perspective challenges in big data[C]//Advances in Intelligent Data Analysis and Applications: Proceeding of the Sixth Euro-China Conference on Intelligent Data Analysis and Applications, 15-18 October 2019, Arad, Romania. 2022: 309-325.
- [4] 中国信通院. 数据要素与先进存储融合发展研究报告 (2024 年) [EB/OL].(2024-09-12)[2025-02-02].https://www.caict.ac.cn/kxyj/qwfb/ztbg/202409/t20240926_493877.htm.
- [5] 国家互联网信息办公室. 国家信息化发展报告 (2023 年) [EB/OL].(2024-09-06)[2025-02-02].https://www.cac.gov.cn/2024-09/06/c_1727308607362592.htm.
- [6] 国家数据局. 数字中国发展报告 (2023 年) [EB/OL].(2024-06-30)[2025-01-25].https://www.nda.gov.cn/sjj/ywpc/sjzg/0830/20240830180401077761745_pc.html.
- [7] 赛迪顾问股份有限公司. 中国分布式存储市场研究报告 (2024) [EB/OL].(2024-06-25)[2025-02-12]. <https://www.mtx.cn/#/report?id=10021709>.
- [8] 中华人民共和国工业和信息化部. 算力基础设施高质量发展行动计划 [EB/OL].(2023-10-08)[2025-02-12].https://www.miit.gov.cn/jgsj/txs/wjfb/art/2023/art_ed448f60021741729f7ee8e36aaafdd7.html.
- [9] Shvachko K, Kuang H, Radia S, et al. The hadoop distributed file system[C]//2010 IEEE 26th symposium on mass storage systems and technologies (MSST). IEEE, 2010: 1-10.
- [10] Sa W, Sa B, El M, et al. Ceph: A scalable, high-performance distributed file system[C]//Proceedings of the 7th Conference on Operating Systems Design and Implementation (OSDI'06). 2006: 307-320.
- [11] Ghemawat S, Gobioff H, Leung S-T. The Google file system[C]//Proceedings of the nineteenth ACM symposium on Operating systems principles. 2003: 29-43.
- [12] 张永福. 分布式数据存储关键技术研究 [D]. 南京邮电大学, 2016.
- [13] 陈豪钧. 浅谈 Server San 实现的软件定义存储 [J]. 信息通信, 2017, (3): 142-143.
- [14] Vishwanath K V, Nagappan N. Characterizing cloud computing hardware reliability[C]//Proceedings of the 1st ACM symposium on Cloud computing. 2010: 193-204.
- [15] Xin Q, Miller E L, Schwarz T, et al. Reliability mechanisms for very large storage systems[C]//20th IEEE/11th NASA Goddard Conference on Mass Storage Systems and Technologies, 2003.(MSST 2003). Proceedings. IEEE, 2003: 146-156.
- [16] Tamim I, Saci A, Jammal M, et al. Downtime-aware o-ran vnf deployment strategy for optimized self-healing in the o-cloud[C]//2021 IEEE Global Communications Conference (GLOBECOM).

- IEEE, 2021: 1-6.
- [17] Sathiamoorthy M, Asteris M, Papailiopoulos D, et al. Xoring elephants: Novel erasure codes for big data[EB/OL].(2013-06-16)[2025-03-10]. <https://arxiv.org/abs/1301.3791>.
- [18] Shannon C E. A mathematical theory of communication[J]. The Bell System Technical Journal, 1948, 27(3): 379-423.
- [19] Costello D J, Forney G D. Channel coding: The road to channel capacity[J]. Proceedings of the IEEE, 2007, 95(6): 1150-1177.
- [20] Reed I S, Solomon G. Polynomial codes over certain finite fields[J]. Journal of the Society for Industrial and Applied Mathematics, 1960, 8(2): 300-304.
- [21] Thomasian A. Storage Systems: Organization, Performance, Coding, Reliability, and Their Data Processing [M]. Academic Press, 2021.
- [22] Kumar G, Dukkipati N, Jang K, et al. Swift: Delay is simple and effective for congestion control in the datacenter[C]//Proceedings of the Annual conference of the ACM Special Interest Group on Data Communication on the applications, technologies, architectures, and protocols for computer communication. 2020: 514-528.
- [23] Arafa Y, Barai A, Zheng M, et al. Fault tolerance performance evaluation of large-scale distributed storage systems HDFS and Ceph case study[C]//2018 IEEE High Performance Extreme Computing Conference (HPEC). IEEE, 2018: 1-7.
- [24] Detchart J, Lacan J. Improving the coding speed of erasure codes with polynomial ring transforms[C]//GLOBECOM 2017-2017 IEEE Global Communications Conference. IEEE, 2017: 1-6.
- [25] Luby M, Zuckermank D. An xor-based erasure-resilient coding scheme[J]. Tech Report, Tech Rep, 1995, 103(508): 9-12.
- [26] 唐聃, 蔡红亮, 耿微. RS 类纠删码的译码方法 [J]. 计算机研究与发展, 2022, 59(03): 582-596.
- [27] Qiu J, Chen L, Liu S. A novel concatenated coding scheme: RS-SC-LDPC codes[J]. IEEE Communications Letters, 2020, 24(10): 2092-2095.
- [28] Lin S J, Al-Naffouri T Y, Han Y S. FFT algorithm for binary extension finite fields and its application to Reed–Solomon codes[J]. IEEE Transactions on Information Theory, 2016, 62(10): 5343-5358.
- [29] Lin S J, Al-Naffouri T Y, Han Y S, et al. Novel polynomial basis with fast Fourier transform and its application to Reed–Solomon erasure codes[J]. IEEE Transactions on Information Theory, 2016, 62(11): 6284-6299.
- [30] Yu L, Lin Z, Lin S J, et al. Fast encoding algorithms for Reed–Solomon codes with between four and seven parity symbols[J]. IEEE Transactions on Computers, 2020, 69(5): 699-705.
- [31] Yu L, Lin S J, Hou H, et al. Reed-Solomon coding algorithms based on Reed-Muller transform for any number of parities[J]. IEEE Transactions on Computers, 2023, 72(9): 2677-2688.
- [32] Deng G, Jiang Z, Bai B, et al. New Reed-Solomon Codes with All XOR Operation for Better En/Decoding Performance[C]//2024 IEEE Information Theory Workshop (ITW). IEEE, 2024: 401-406.
- [33] 孙怡宁, 黄秋, 胡剑浩. Reed-Solomon 码概率软译码算法增强技术 [J]. 中国科学: 信息科学,

- 2021, 51(8): 1331.
- [34] Tang N, Han Y S. A new decoding method for Reed–Solomon codes based on FFT and modular approach[J]. IEEE Transactions on Communications, 2022, 70(12): 7790-7801.
- [35] Blomer J. An XOR-based erasure-resilient coding scheme[J]. Technical Report at ICSI, 1995, 103 (508): 9-12.
- [36] Tang Y J, Zhang X. Fast en/decoding of Reed-Solomon codes for failure recovery[J]. IEEE Transactions on Computers, 2021, 71(3): 724-735.
- [37] Liu W, Jiang Z, Huang Z, et al. On the I/O Cost of Linear Repair Schemes for Arbitrary (n, k) Reed-Solomon Codes[C]//2024 IEEE Information Theory Workshop (ITW). IEEE, 2024: 253-258.
- [38] Dimakis A G, Godfrey P B, Wu Y, et al. Network coding for distributed storage systems[J]. IEEE Transactions on Information theory, 2010, 56(9): 4539-4551.
- [39] Hou H, Lee P P C, Shum K W, et al. Rack-aware regenerating codes for data centers[J]. IEEE Transactions on Information Theory, 2019, 65(8): 4730-4745.
- [40] Hou H, Lee P P C. Generalized rack-aware regenerating codes for jointly optimal node and rack repairs[C]//2021 IEEE International Symposium on Information Theory (ISIT). IEEE, 2021: 2191-2196.
- [41] Blaum M, Farrell P G, Van Tilborg H C, et al. Array codes[J]. Handbook of Coding Theory, 1998, 2: 1855-1909.
- [42] Blaum M, Brady J, Bruck J, et al. EVENODD: An efficient scheme for tolerating double disk failures in RAID architectures[J]. IEEE Transactions on Computers, 1995, 44(2): 192-202.
- [43] Corbett P, English B, Goel A, et al. Row-diagonal parity for double disk failure correction[C]//Proceedings of the 3rd USENIX Conference on File and Storage Technologies. 2004: 1-14.
- [44] Blaum M, Bruck J, Vardy A. MDS array codes with independent parity symbols[J]. IEEE Transactions on Information Theory, 1996, 42(2): 529-542.
- [45] Blaum M, Brady J, Bruck J, et al. The EVENODD code and its generalization[J]. High Performance Mass Storage and Parallel I/O, 2001: 187-208.
- [46] Huang S, Hou H, Yu X. A lower bound on disk reads for single information disk failure recovery and one recovery scheme for EVENODD $(p, 3)$ [C]//2019 Ninth International Workshop on Signal Design and its Applications in Communications (IWSDA). IEEE, 2019: 1-5.
- [47] Goel A, Corbett P. RAID triple parity[J]. ACM SIGOPS Operating Systems Review, 2012, 46(3): 41-49.
- [48] Haibo J, Mingyu F, Yilong X, et al. Improved decoding algorithm for the generalized EVENODD array code[C]//Proceedings of 2012 2nd International Conference on Computer Science and Network Technology. IEEE, 2012: 2216-2219.
- [49] Hou H, Lee P P C, Han Y S, et al. Triple-fault-tolerant binary MDS array codes with asymptotically optimal repair[C]//2017 IEEE International Symposium on Information Theory (ISIT). IEEE, 2017: 839-843.
- [50] Lan Y, Hou H, Zhang P. Efficient Repair Algorithm for Information Column of EVENODD $(p, 4)$

- Codes[C]//2020 IEEE 20th International Conference on Communication Technology (ICCT). IEEE, 2020: 447-453.
- [51] Hou H, Lee P P C. A new construction of EVENODD codes with lower computational complexity[J]. IEEE Communications Letters, 2018, 22(6): 1120-1123.
- [52] Fu H, Hou H, Zhang L. Extended EVENODD+ codes with asymptotically optimal updates and efficient encoding/decoding[C]//2021 XVII International Symposium" Problems of Redundancy in Information and Control Systems"(REDUNDANCY). IEEE, 2021: 1-6.
- [53] Chen R, Xu L. A new decoding algorithm for XOR-based erasure codes[J]. SN Computer Science, 2020, 1: 1-19.
- [54] Wan W, Wu Z, Chen Y, et al. A data placement based on toleration on triple failures array codes in RAID[J]. Chinese Journal Of Computers-Chinese Edition, 2007, 30(10): 1721.
- [55] Huang C, Xu L. STAR: An efficient coding scheme for correcting triple storage node failures[J]. IEEE Transactions on Computers, 2008, 57(7): 889-901.
- [56] Hou H, Shum K W, Chen M, et al. New MDS array code correcting multiple disk failures[C]//2014 IEEE Global Communications Conference. IEEE, 2014: 2369-2374.
- [57] Zhu P, Lv J, Han Y S, et al. New EVENODD+ Codes with More Flexible Parameters and Lower Complexity[C]//2024 IEEE International Symposium on Information Theory (ISIT). IEEE, 2024: 1233-1238.
- [58] Hou H, Han Y S. A new construction and an efficient decoding method for Rabin-like codes[J]. IEEE Transactions on Communications, 2017, 66(2): 521-533.
- [59] Wan S, Cao Q, Xie C, et al. Code-m: A non-mds erasure code scheme to support fast recovery from up to two-disk failures in storage systems[C]//2010 IEEE/IFIP International Conference on Dependable Systems & Networks (DSN). IEEE, 2010: 51-60.
- [60] Hafner J L. WEAVER Codes: Highly Fault Tolerant Erasure Codes for Storage Systems[C]//FAST. 2005, 5: 211-224.
- [61] Xu L, Bruck J. X-code: MDS array codes with optimal encoding[J]. IEEE Transactions on Information Theory, 1999, 45(1): 272-276.
- [62] Jin C, Jiang H, Feng D, et al. P-Code: A new RAID-6 code with optimal properties[C]//Proceedings of the 23rd international conference on Supercomputing. 2009: 360-369.
- [63] Papailiopoulos D S, Dimakis A G. Locally repairable codes[J]. IEEE Transactions on Information Theory, 2014, 60(10): 5843-5855.
- [64] Mullen G L, Panario D. Handbook of Finite Fields [M]. CRC press Boca Raton, 2013.
- [65] Carlitz L. The arithmetic of polynomials in a Galois field[J]. American Journal of Mathematics, 1932, 54(1): 39-50.
- [66] Radmanović M M, Gajić D B, Stanković R S. Efficient computation of Galois field expressions on hybrid CPU-GPU platforms[J]. Journal of Multiple-Valued Logic & Soft Computing, 2016, 26.
- [67] 余蕾蕾. Reed-Solomon 码的快速算法设计与实现 [D]. 中国科学技术大学, 2021.
- [68] Fedorenko S V. Normalized cyclic convolution: The case of even length[J]. IEEE Transactions on Signal Processing, 2015, 63(20): 5307-5317.

- [69] Blahut R E. Theory and practice of error control codes[J]. Reading, MA: Addison-Wesley, 1983, 483-492.
- [70] Yang S. On the LU factorization of the Vandermonde matrix[J]. Discrete Applied Mathematics, 2005, 146(1): 102-105.
- [71] Hou H, Han Y S, Shum K W, et al. A unified form of EVENODD and RDP codes and their efficient decoding[J]. IEEE Transactions on Communications, 2018, 66(11): 5053-5066.
- [72] Hu Y, Chen H C, Lee P P C, et al. NCCloud: applying network coding for the storage repair in a cloud-of-clouds[C]//FAST. 2012, 21.
- [73] Plank J, Greenan K. Jerasure: A library in C facilitating erasure coding for storage applications version 2.0. Technical Report UT-EECS-14-721[J]. University of Tennessee, 2014.
- [74] Cooperation I. Intel intelligent storage acceleration library[EB/OL].(2016-07-08)[2025-02-05]. <https://github.com/spdk/isa-l>.



學而知不足 楊振宇

